CO3020 - Computer Organization 計算機組織

Arithmetic for Computers

電腦的運算

江育綺

2026-10-21



NCU Communication Engineering

Outline

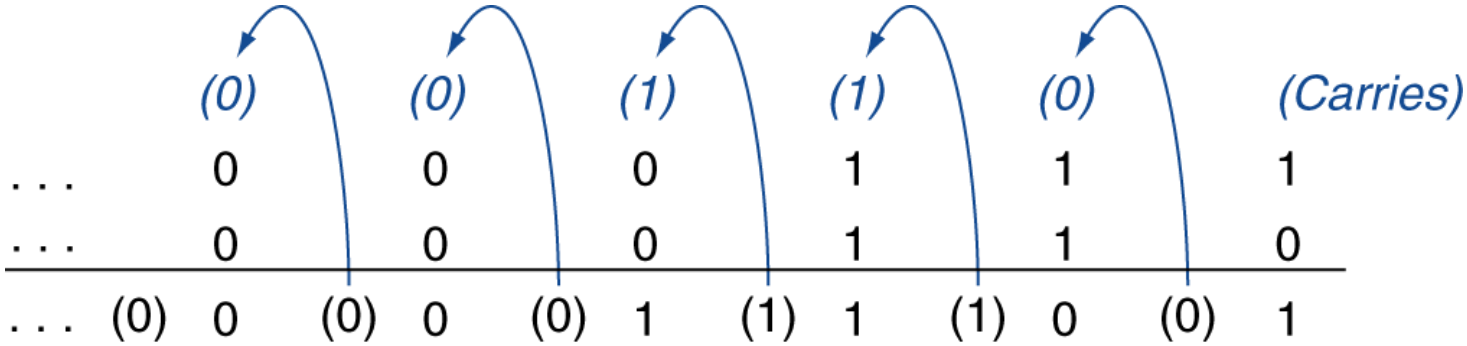
- Operations on integers
  - Addition and Subtraction
  - Constructing an Arithmetic Logic Unit (ALU)
    - Add, sub, and, or, nor
    - Set-on-less-than
    - Overflow Detection, Zero Detection
  - Multiplication
  - Division
- Floating-point real numbers
  - Floating Point

Addition and Subtraction

CHAPTER 3: ARITHMETIC FOR COMPUTERS

Integer Addition

- Example: 7 + 6



Integer Subtraction

- Add negation of second operand
- Example: 7 – 6 = 7 + (–6)
  - +7: 0000 0000 … 0000 0111
    –6: 1111 1111 … 1111 1010
    +1: 0000 0000 … 0000 0001
- Overflow if result out of range
  - Adding +ve and –ve operands, no overflow
  - Adding two +ve operands
    - Overflow if result sign is 1
  - Adding two –ve operands
    - Overflow if result sign is 0

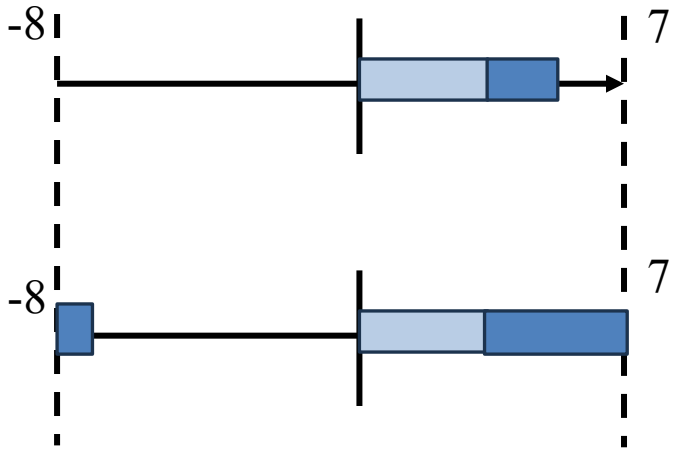
*carry: ignored

2's Complement

*example, n=4, range is $-8 \sim 7$

1. Addition of two positive numbers, sum $< 2^{n-1}$

	+3	$(0011)_2$	
	+4	$(0100)_2$	
Positive	+7	$(0111)_2$	➔ Correct



2. Addition of two positive numbers, sum $\geq 2^{n-1}$

	+5	$(0101)_2$	
	+6	$(0110)_2$	
Positive	+11	$(1011)_2$	➔ Wrong, ➔ it should be Positive Overflow (溢位；溢值)

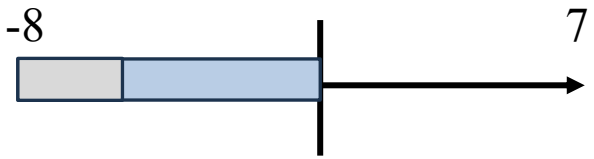
*carry: ignored

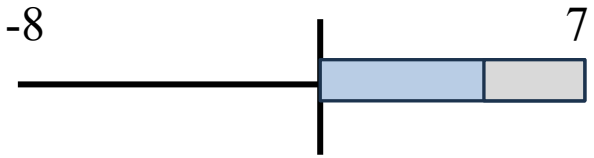
2's Complement

*example, n=4, range is $-8 \sim 7$

3. Addition of positive and negative numbers
(Negative number has greater magnitude)

$$\begin{array}{rrl} & +5 & (0101)_2 \\ & -6 & (1010)_2 \\ \hline \text{Negative} & -1 & (1111)_2 \end{array}$$ ➔ Correct





4. Addition of positive and negative numbers
(Positive number has greater magnitude)

$$\begin{array}{rrl} & -5 & (1011)_2 \\ & +6 & (0110)_2 \\ \hline \text{Positive} & +1 & (1)(0001)_2 \end{array}$$ ➔ Correct

ignored

*carry: ignored

2's Complement

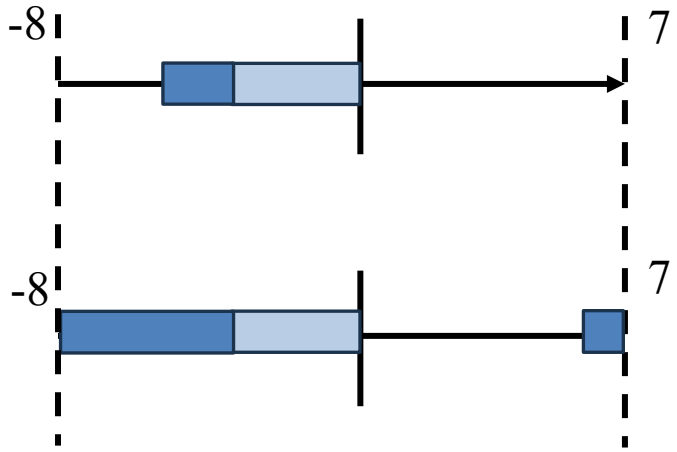
*example, n=4, range is $-8 \sim 7$

5. Addition of two negative numbers, $|\text{sum}| \leq 2^{n-1}$

$$\begin{array}{ll} -3 & (1101)_2 \\ -4 & (1100)_2 \\ \hline & (1)(1001)_2 \end{array}$$

⬇

ignored



6. Addition of two negative numbers, $|\text{sum}| > 2^{n-1}$

$$\begin{array}{ll} -5 & (1011)_2 \\ -6 & (1010)_2 \\ \hline & (1)(0101)_2 \end{array}$$

➔ Wrong,

➔ it should be Negative

Overflow (溢位；溢值)

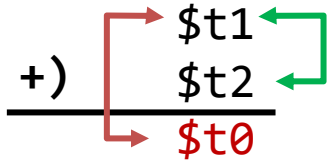
Integer Subtraction

- Add negation of second operand
- Example: 7 – 6 = 7 + (–6)
  - +7: 0000 0000 … 0000 0111
    –6: 1111 1111 … 1111 1010
    +1: 0000 0000 … 0000 0001
- Overflow if result out of range

Op.	Operand A	Operand B	Overflow Result
$A + B$	≥ 0	≥ 0	< 0
	≥ 0	< 0	-
	< 0	≥ 0	-
	< 0	< 0	≥ 0

Op.	Operand A	Operand B	Overflow Result
$A - B$	≥ 0	≥ 0	-
	≥ 0	< 0	< 0
	< 0	≥ 0	≥ 0
	< 0	< 0	-

Signed Addition



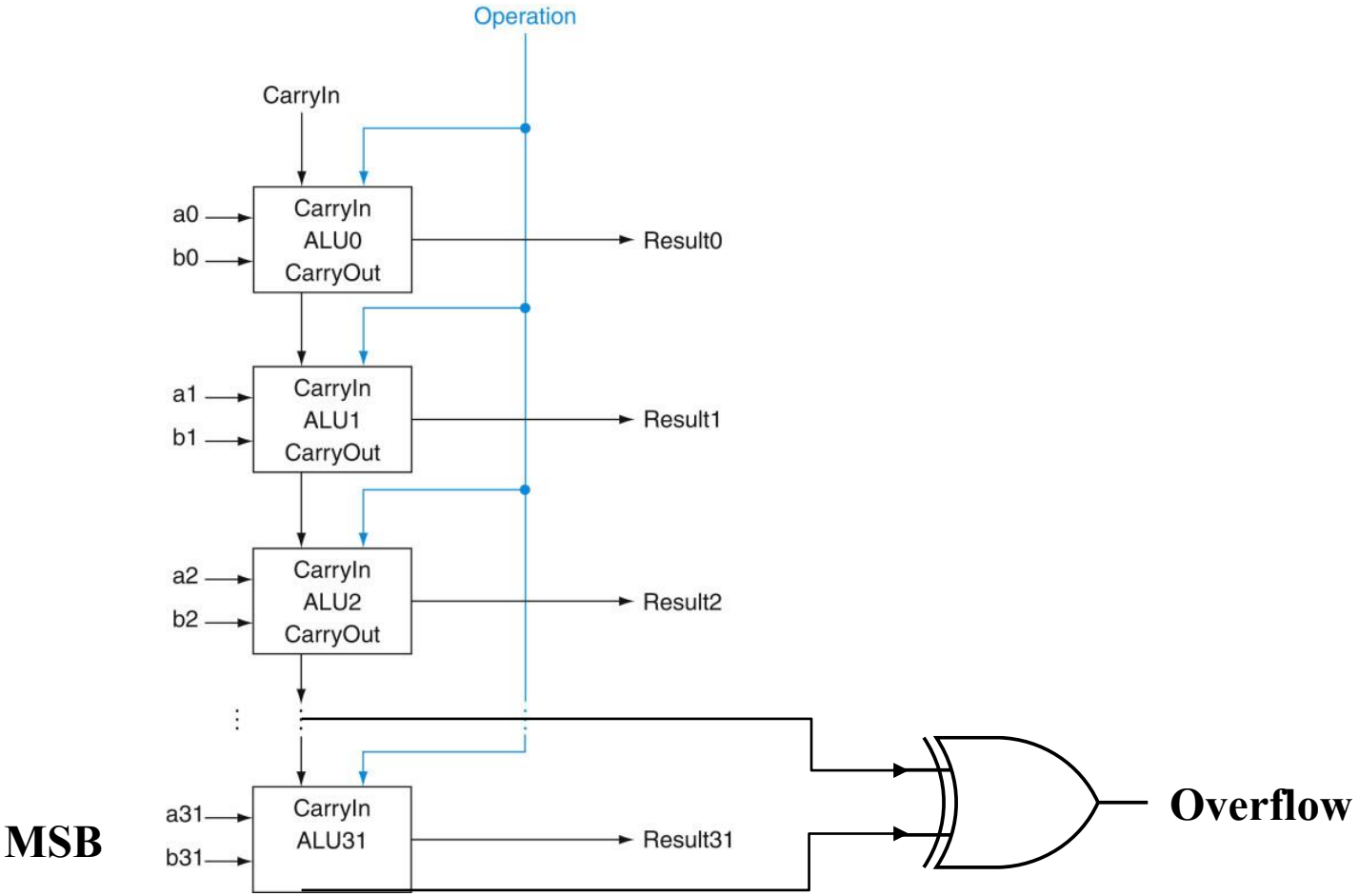

```
addu $t0, $t1, $t2                  # $t0 = $t1 + $t2
xor $t3, $t1, $t2                   # $t3 = $t1 ⊕ $t2
# if signs differ in $t1 and $t2,
# sign bit of $t3 is 1 ➔ negative number
slt $t3, $t3, $zero
            # set $t3 = 1 if XOR result is negative ($t3 < 0)
bne $t3, $zero, No_overflow
            # Different signs in $t1 and $t2 ➔ No overflow.
xor $t3, $t0, $t1                   # $t3 = $t0 ⊕ $t1
# if signs differ in $t0 and $t1,
# sign bit of $t3 is 1 ➔ negative number
slt $t3, $t3, $zero
bne $t3, $zero, Overflow
            # Different signs in $t0 and $t1 ➔ Overflow!
```

Check operand signs $t1/$t2

Check the sum's sign $t0

Signed Overflow Detection

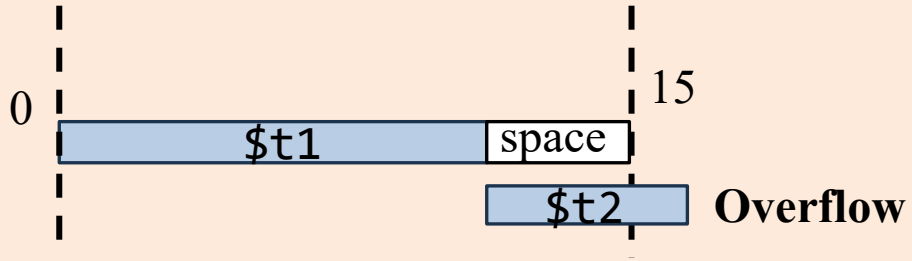
- Overflow if: **Carry into MSB ≠ Carry out of MSB**



Unsigned Addition

```
addu $t0, $t1, $t2          # $t0 = $t1 + $t2
nor $t3, $t1, $zero         # $t3 = !$t1 → (2^n − 1) − $t1
                            # Calculate remaining space
sltu $t3, $t3, $t2          # Set $t3 = 1 if space < $t2
bne $t3, $zero, Overflow    # Overflow!
```

MAX



Dealing with Overflow

- Some languages ignore overflow
  - Use MIPS addu, addui, subu instructions
- Other languages require raising an exception
  - Use MIPS add, addi, sub instructions
  - On overflow, invoke exception handler
    - Save PC in exception program counter (EPC) register
    - Jump to predefined handler address
    - mfc0 (move from coprocessor reg) instruction can retrieve EPC value, to return after corrective action

$k0, $k1 are preserved for OS

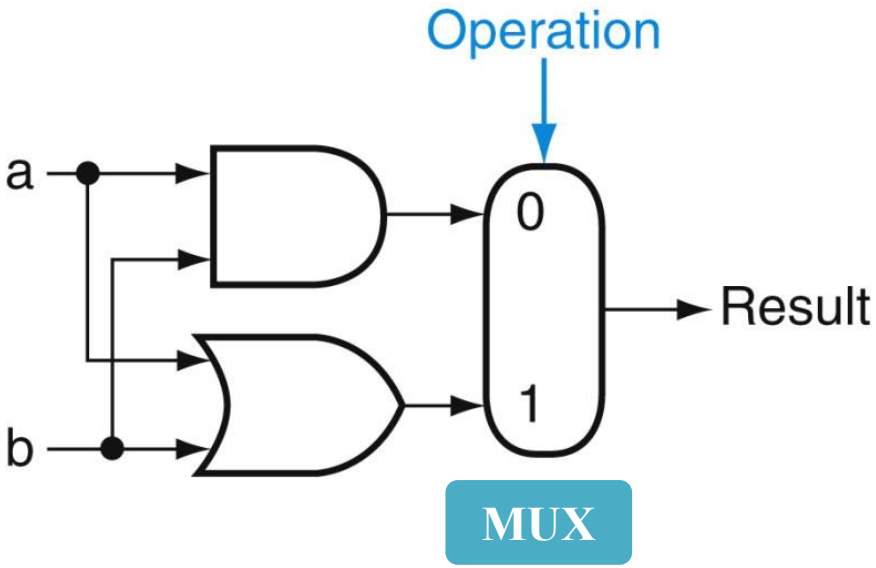
Constructing an Arithmetic Logic Unit (ALU)

CHAPTER 3: ARITHMETIC FOR COMPUTERS

MIPS ALU Designing

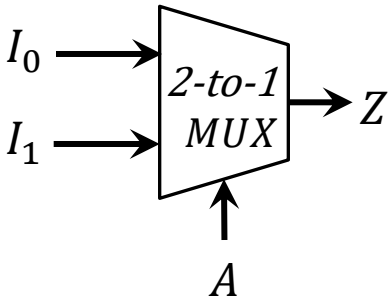
- Must support the following arithmetic and logic operations
  - **Add, sub**: 2's complement adder/subtractor with overflow detection
  - **And, or, nor**: Logical AND, OR, NOR
  - **Slt** (set-on-less-than): 2's complement adder with inverter, check sign bit of result.

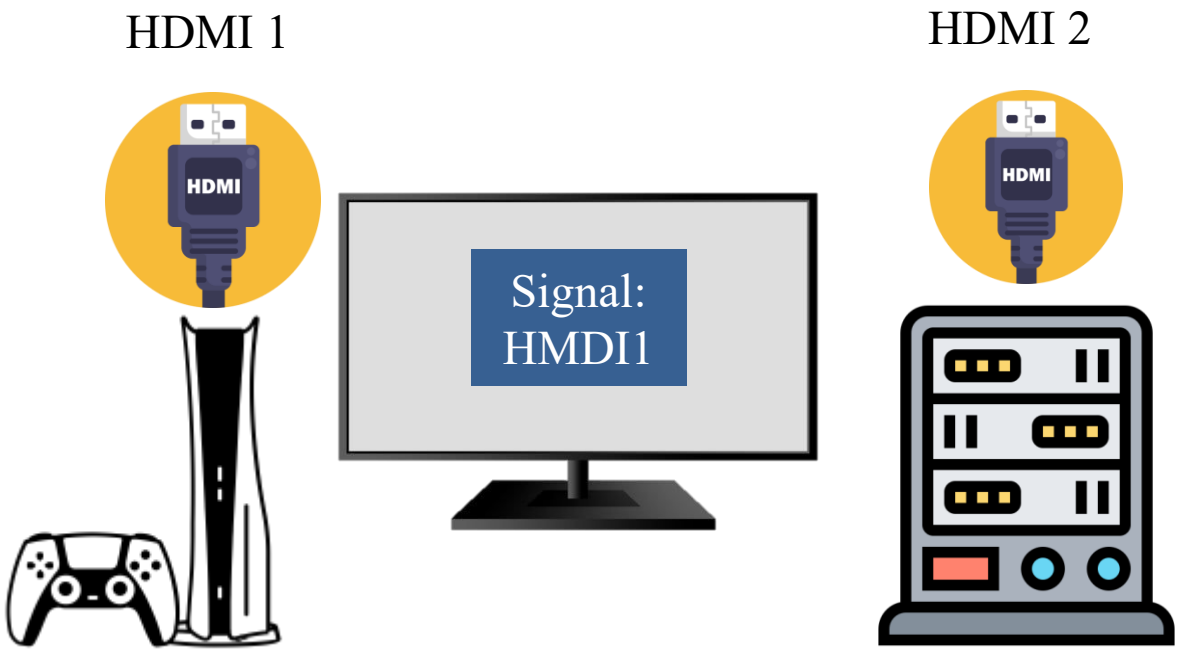
1-bit Logical Unit for AND, OR



Appendix - Multiplexers (MUX)

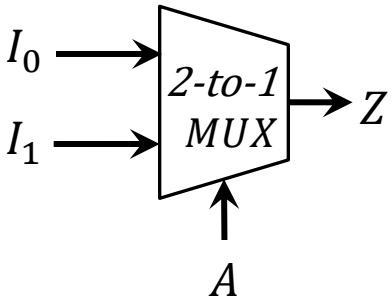
- Multiplexers (多工器)
  - 2-to-1 MUX

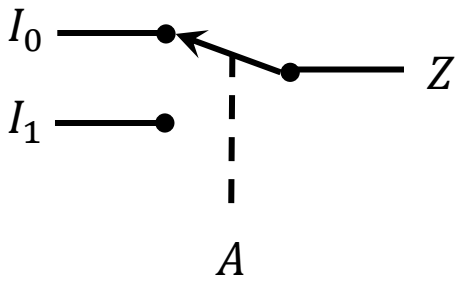


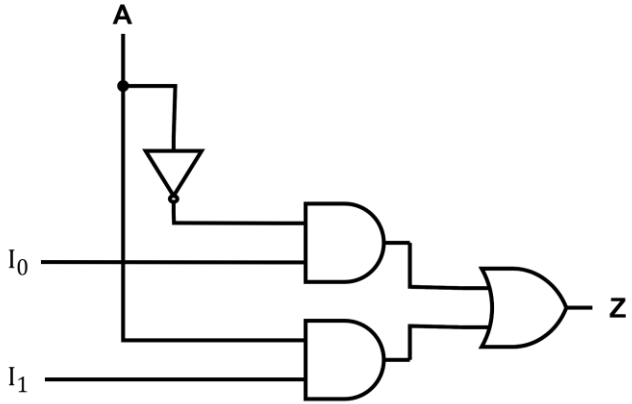


Appendix - Multiplexers (MUX)

- Multiplexers (多工器)
  - 2-to-1 MUX





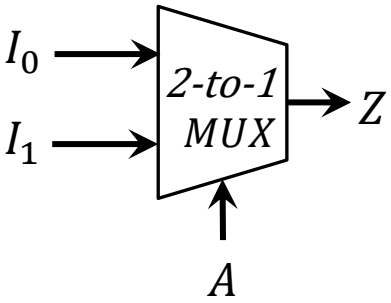


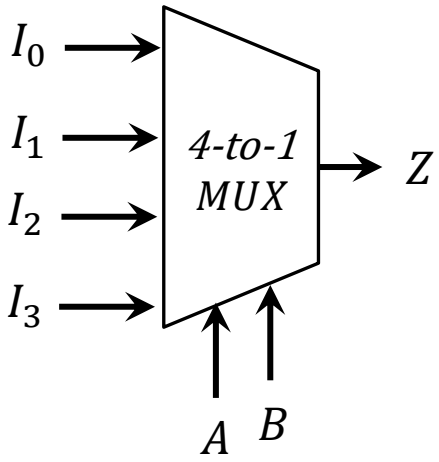
A	Z
0	I_0
1	I_1

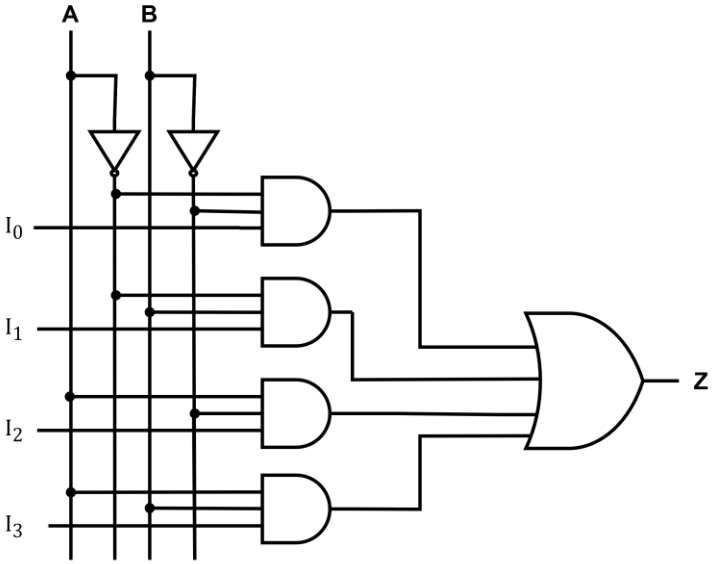
$$Z = A'I_0 + AI_1$$

Appendix - Multiplexers (MUX)

- Multiplexers (多工器)
  - 2-to-1 MUX
  - 4-to-1 MUX

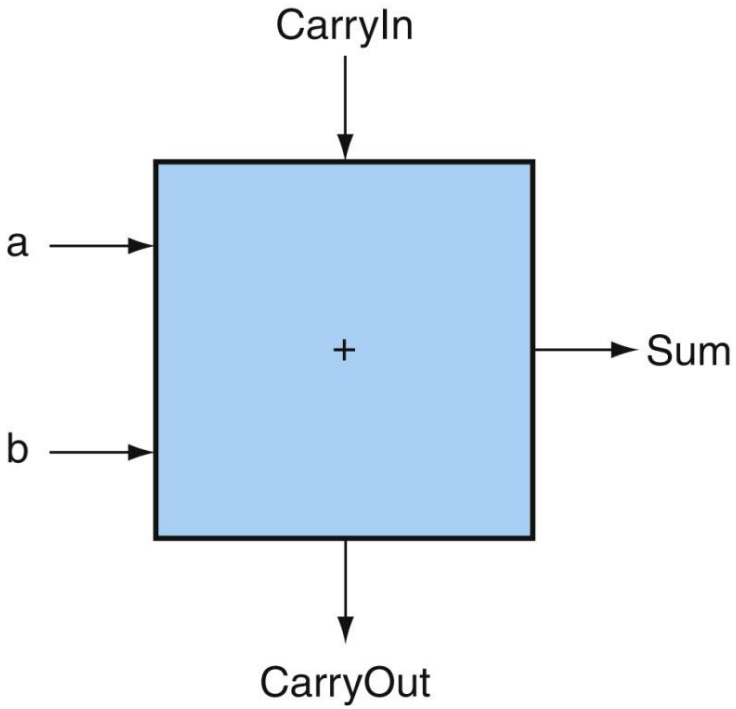




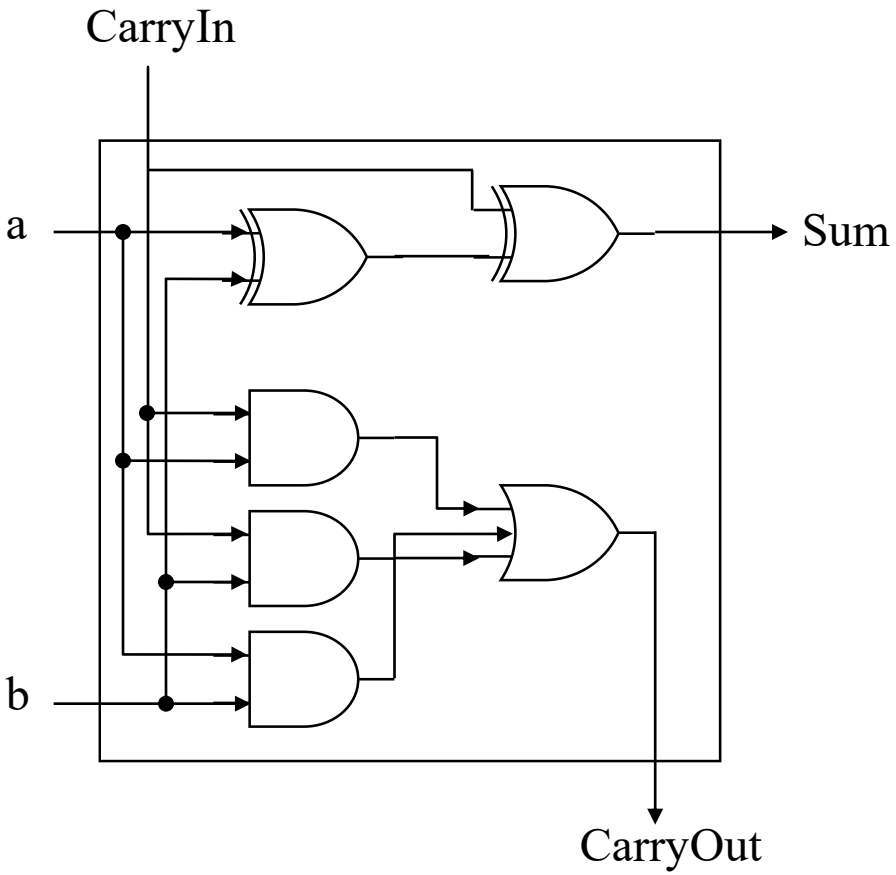


$$Z = A'I_0 + AI_1 \qquad Z = A'B'I_0 + A'BI_1 + AB'I_2 + ABI_3$$

1-bit Adder

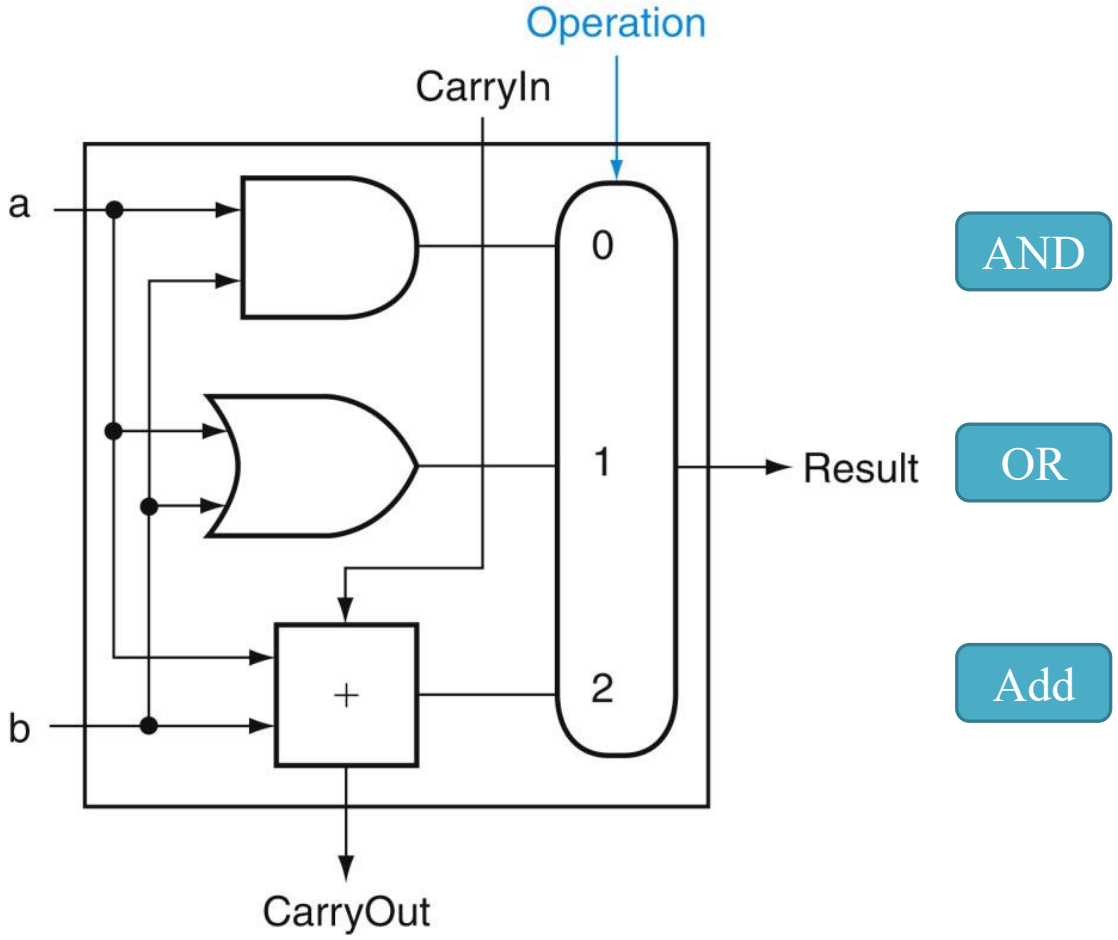


Symbolic

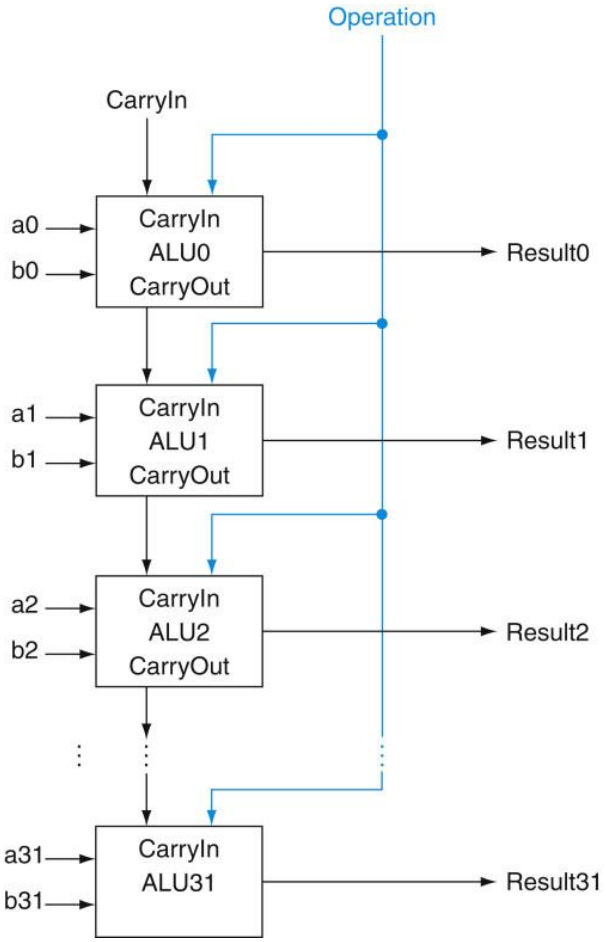


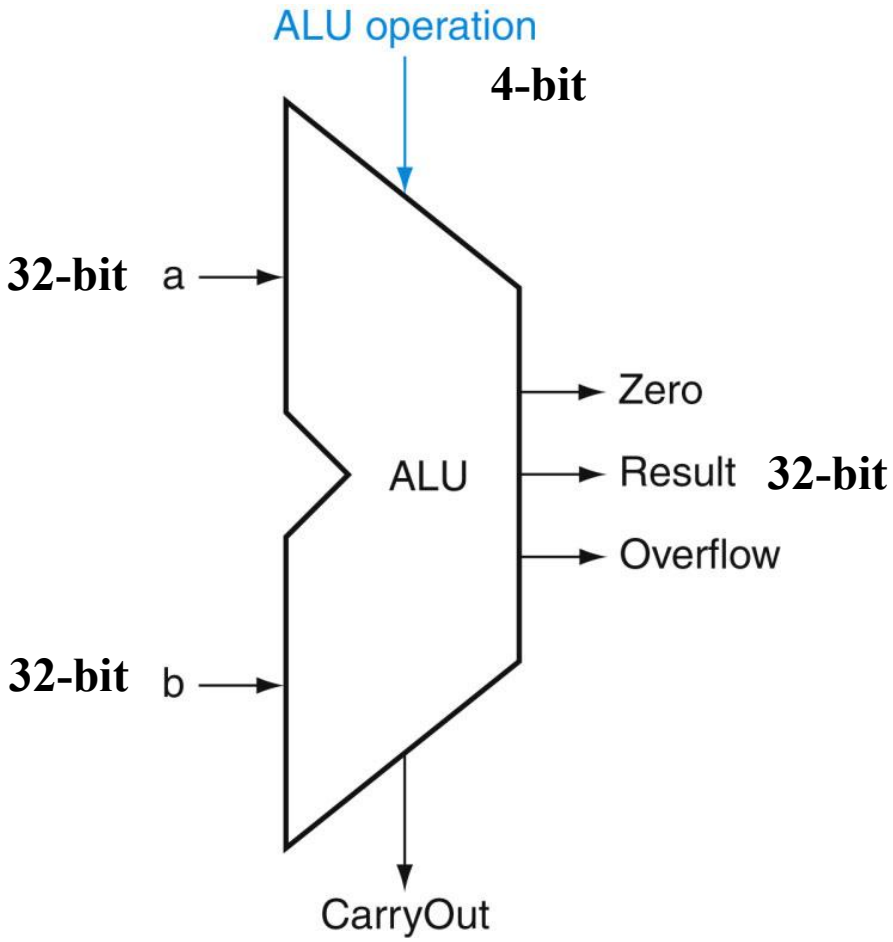
Gate-level Implementation

1-bit ALU

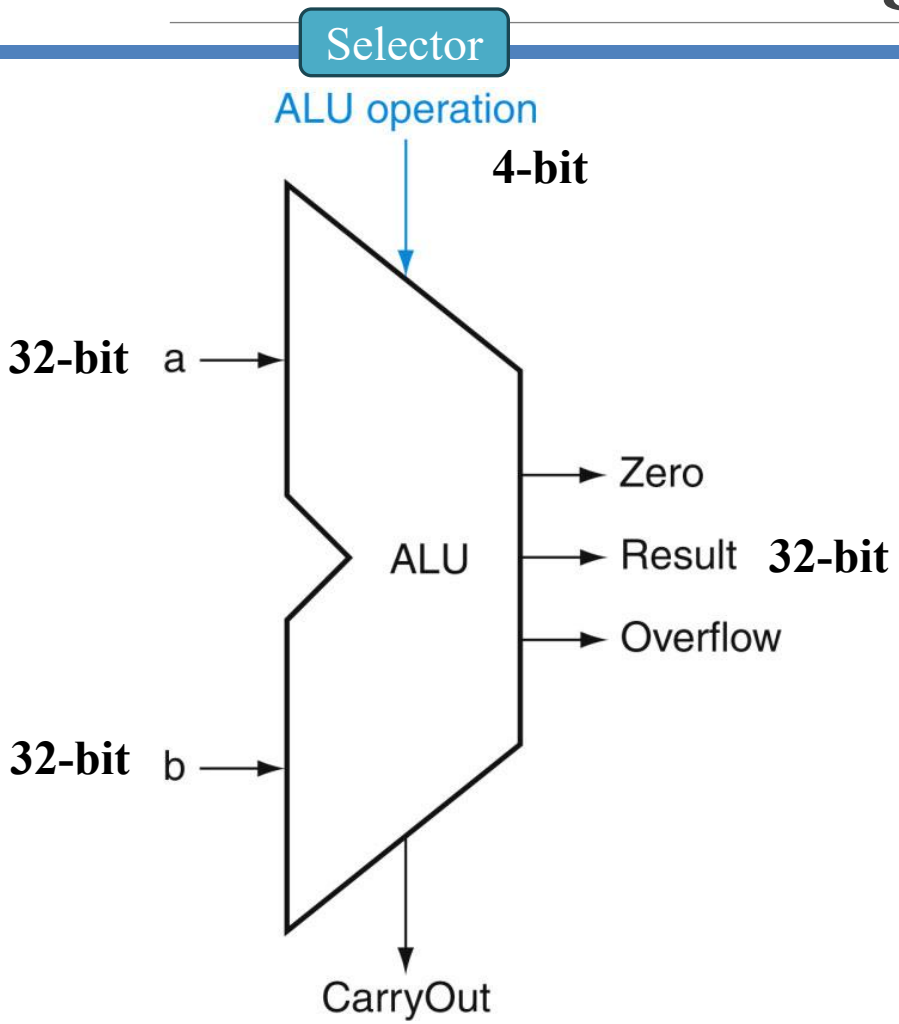


32-bit ALU constructed





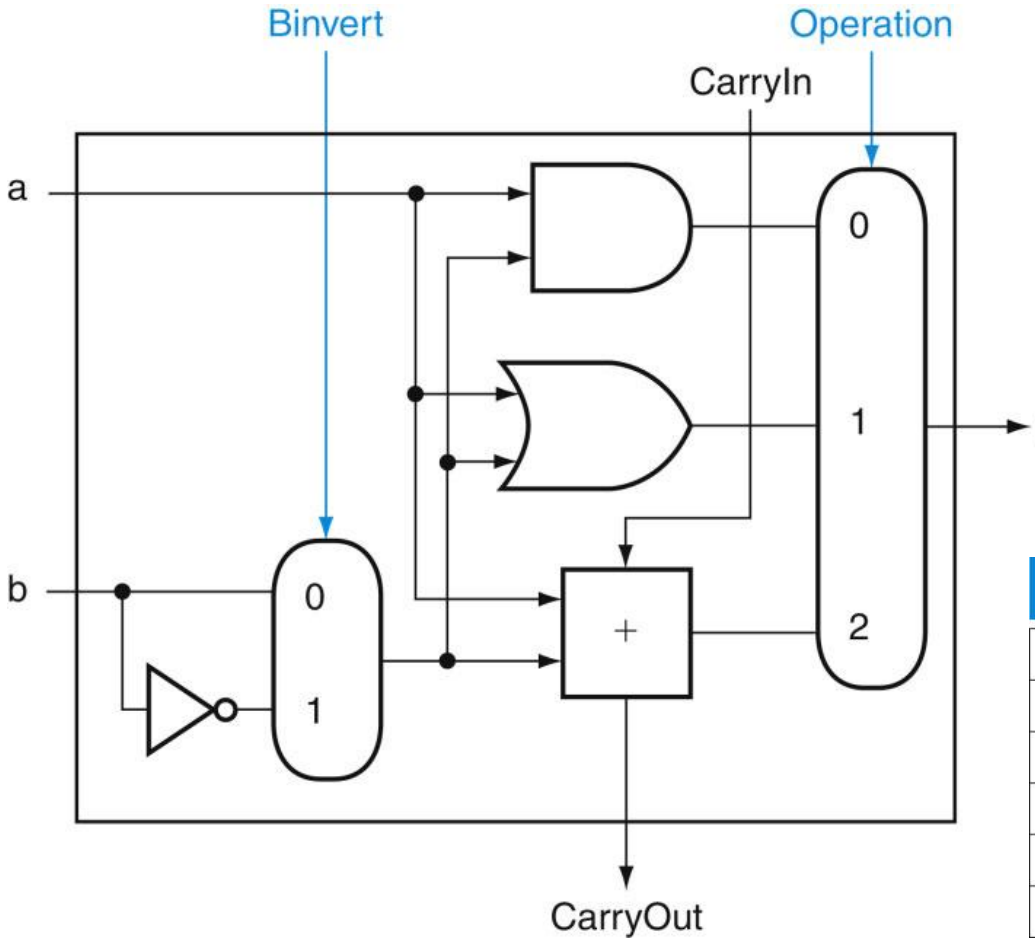
Arithmetic Logic Unit



Selector

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

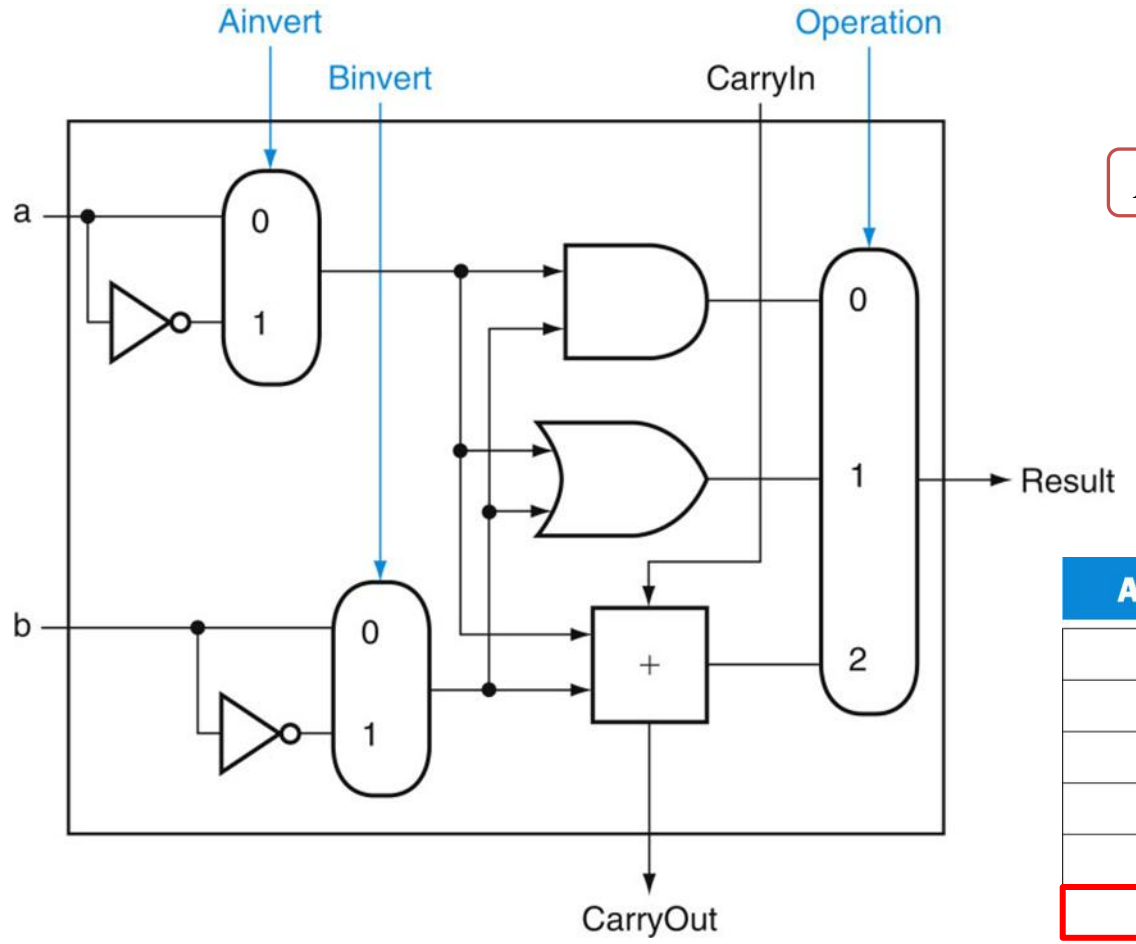
Subtract Operation

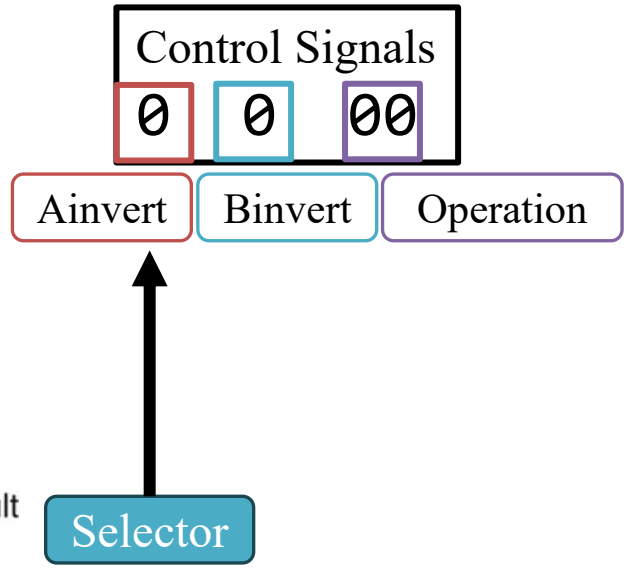


Selector

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

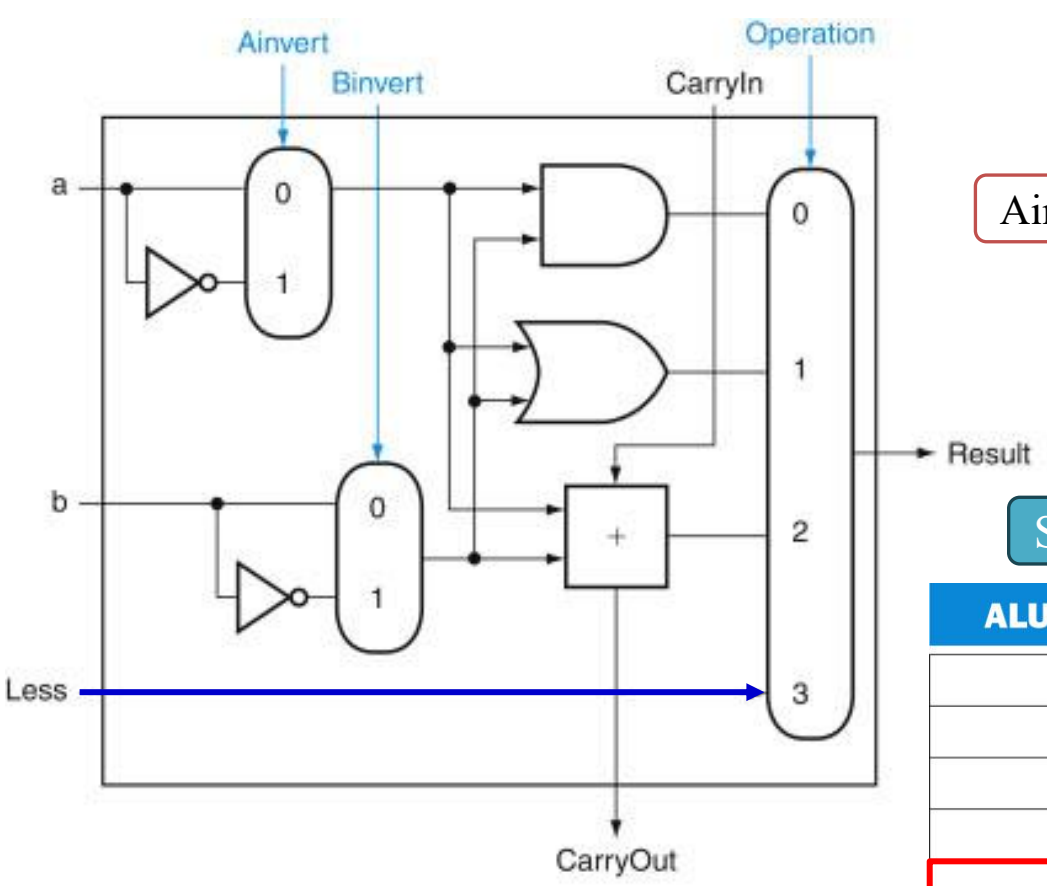
NOR Operation

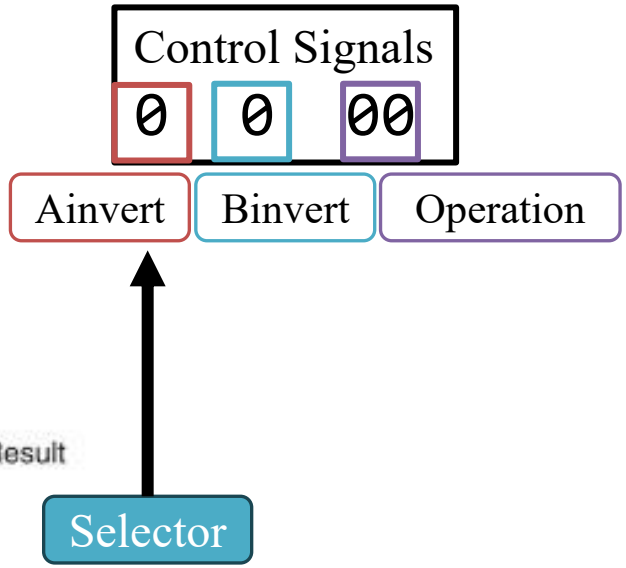




ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

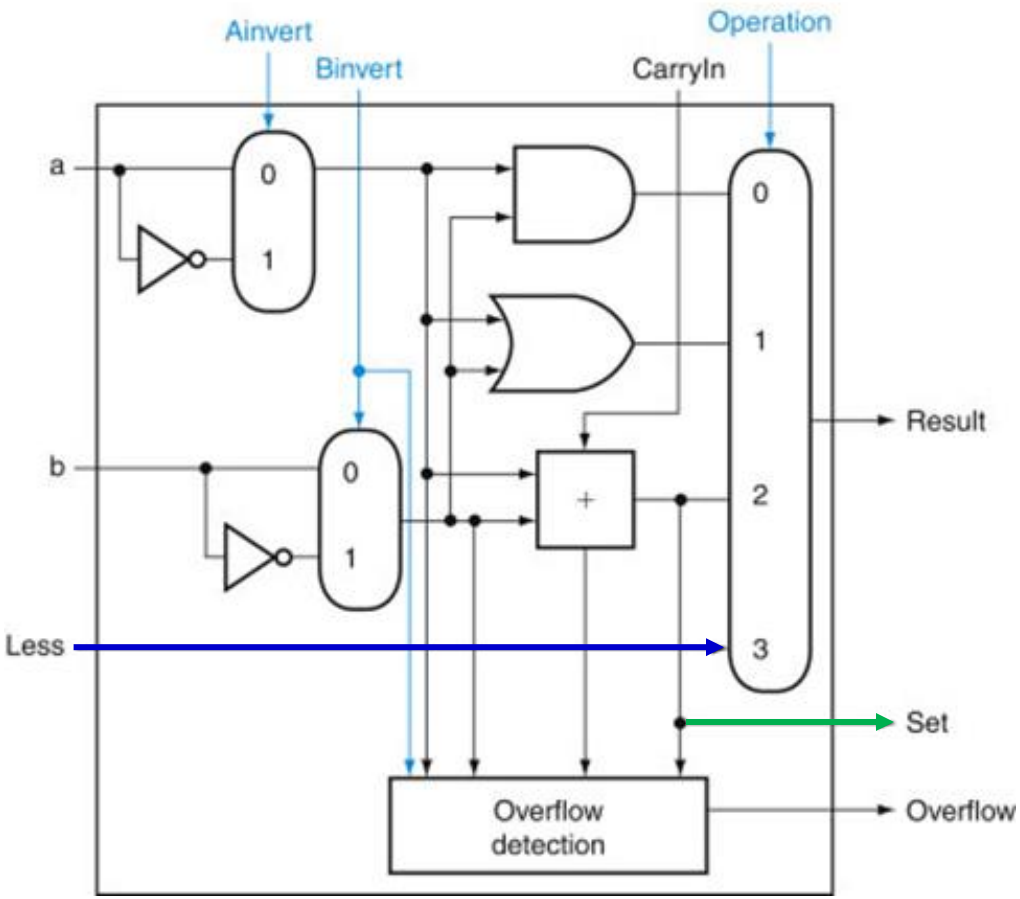
Set-on-less-than (I)

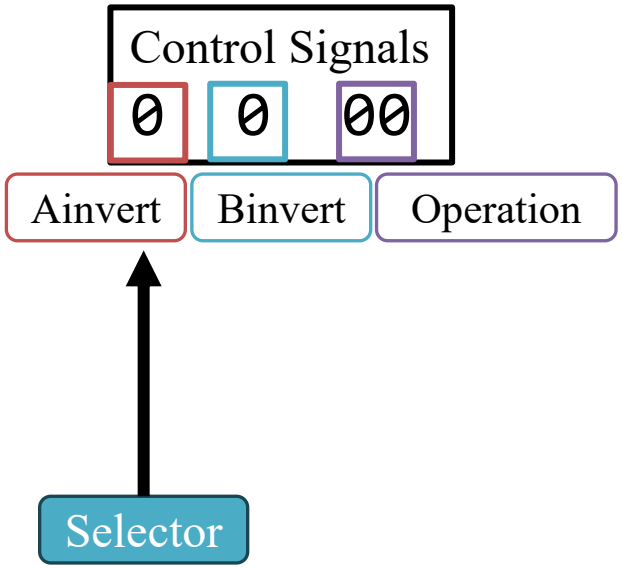




ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

Set-on-less-than (II)

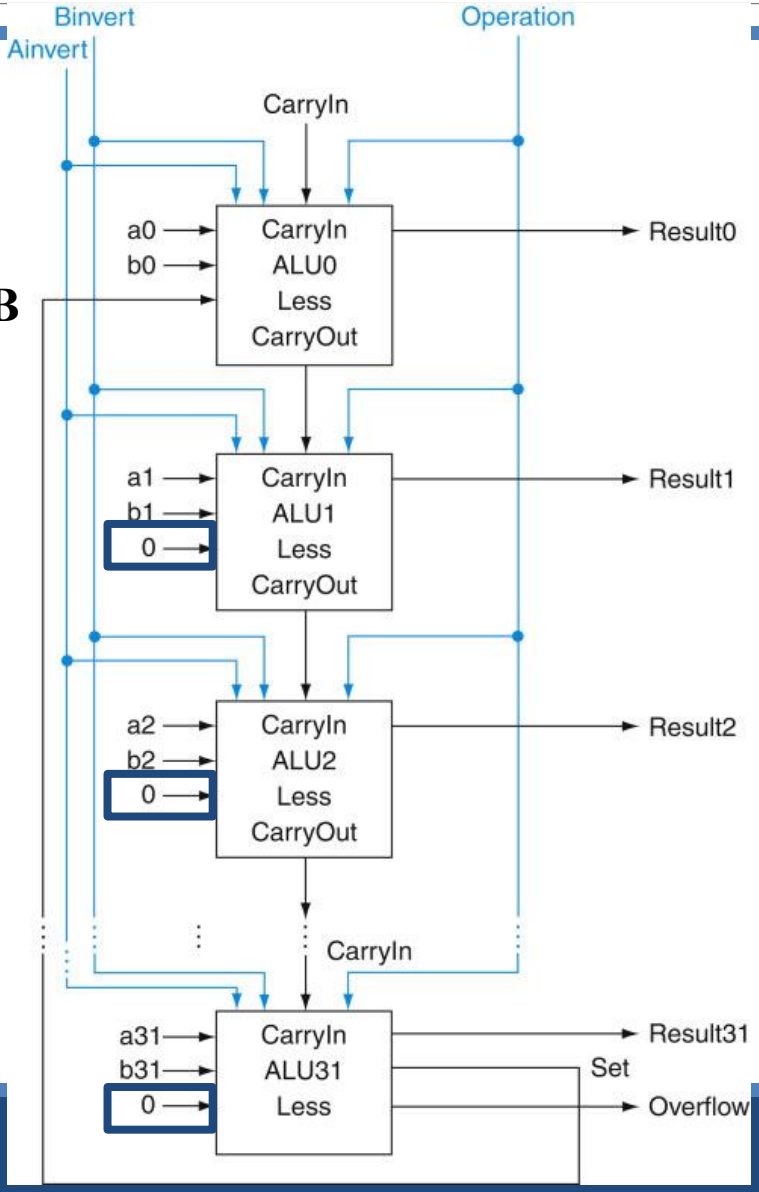




ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

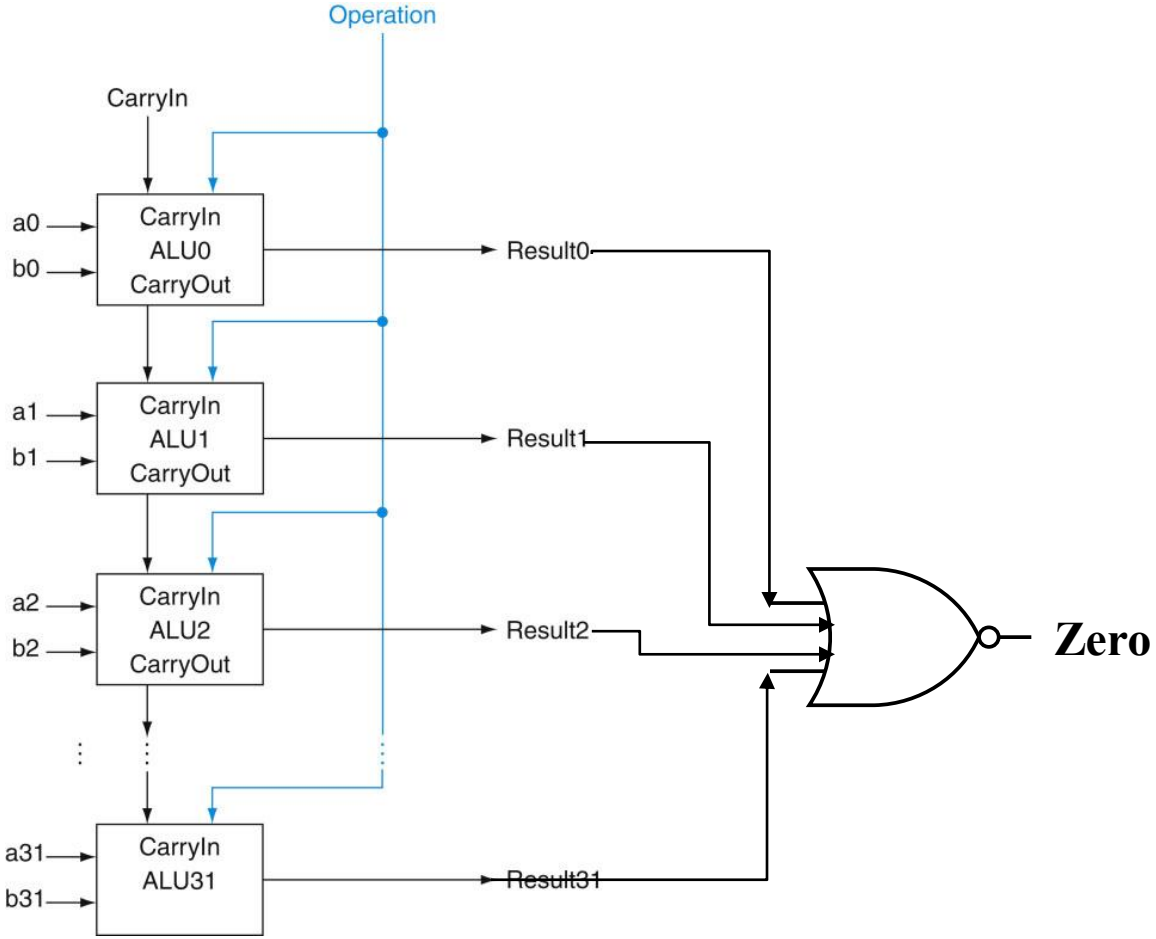
Set-on-less-than (III)

Only set for LSB

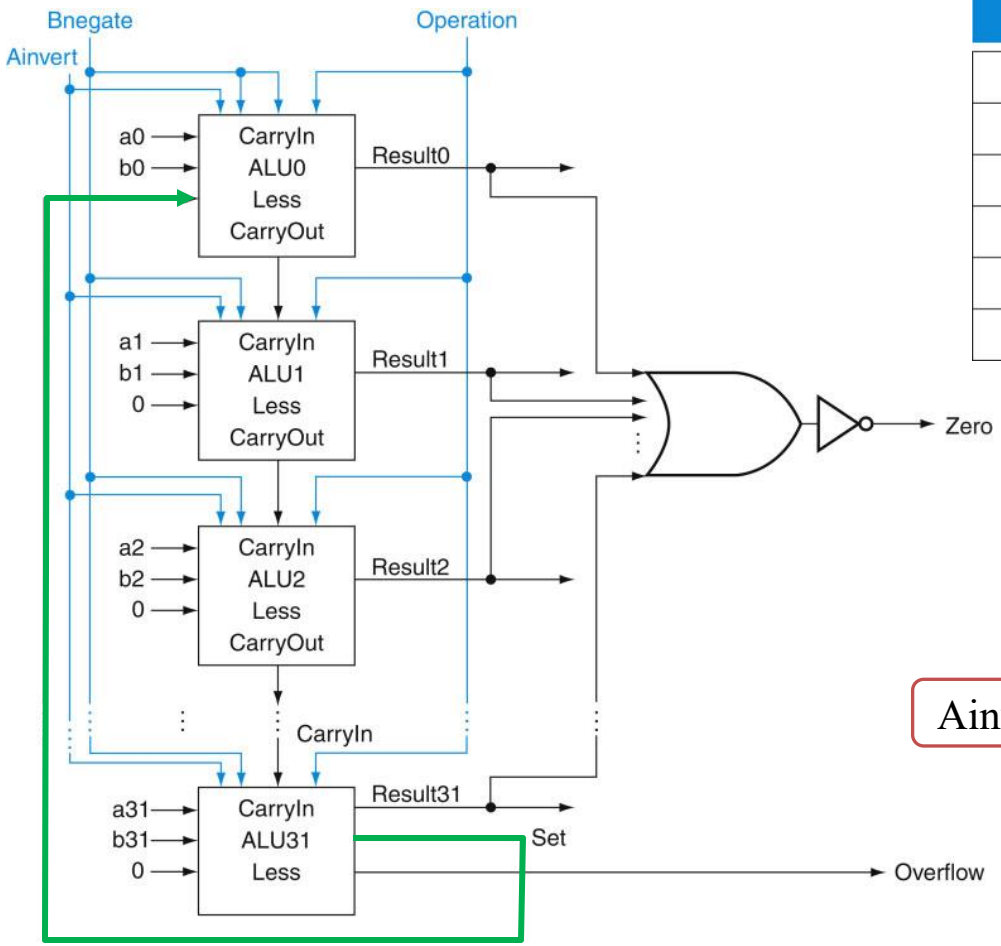


Zero Detection Logic

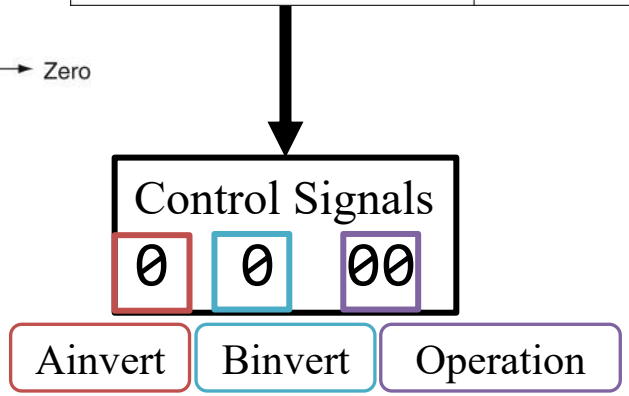
- Zero Detection Logic is a one BIG NOR gate (support conditional jump)



A Final 32-bit ALU



ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR



Multiplication

CHAPTER 3: ARITHMETIC FOR COMPUTERS

Multiplication

- Start with long-multiplication approach

multiplicand

multiplier

partial product

product

```
  1000
× 1001
------
  1000
 0000
00000
1000
-------
1001000
```

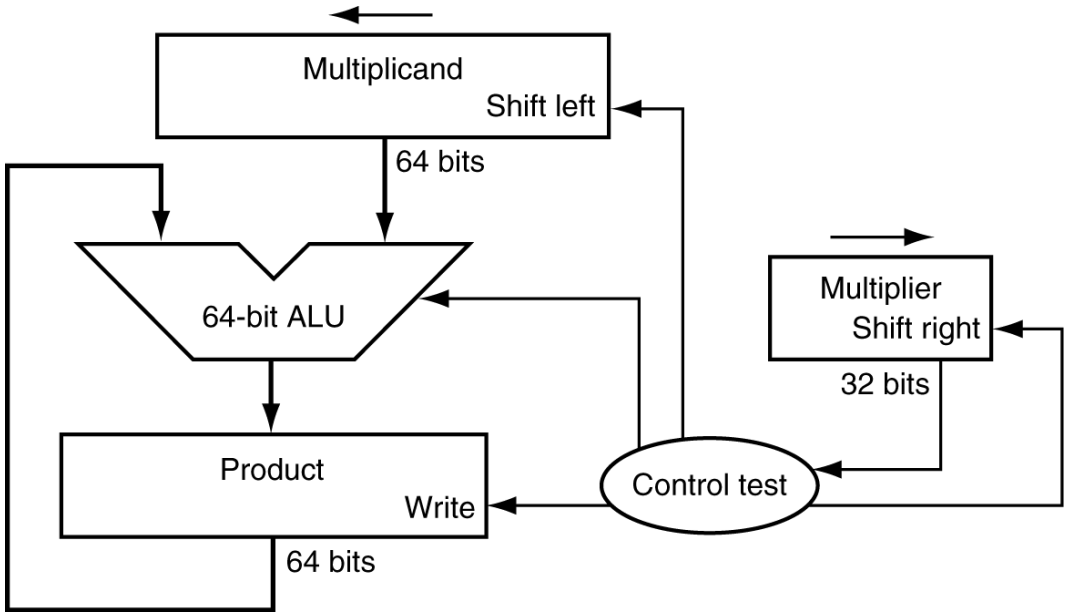
Length of product is the sum of operand lengths

1000 × 1001

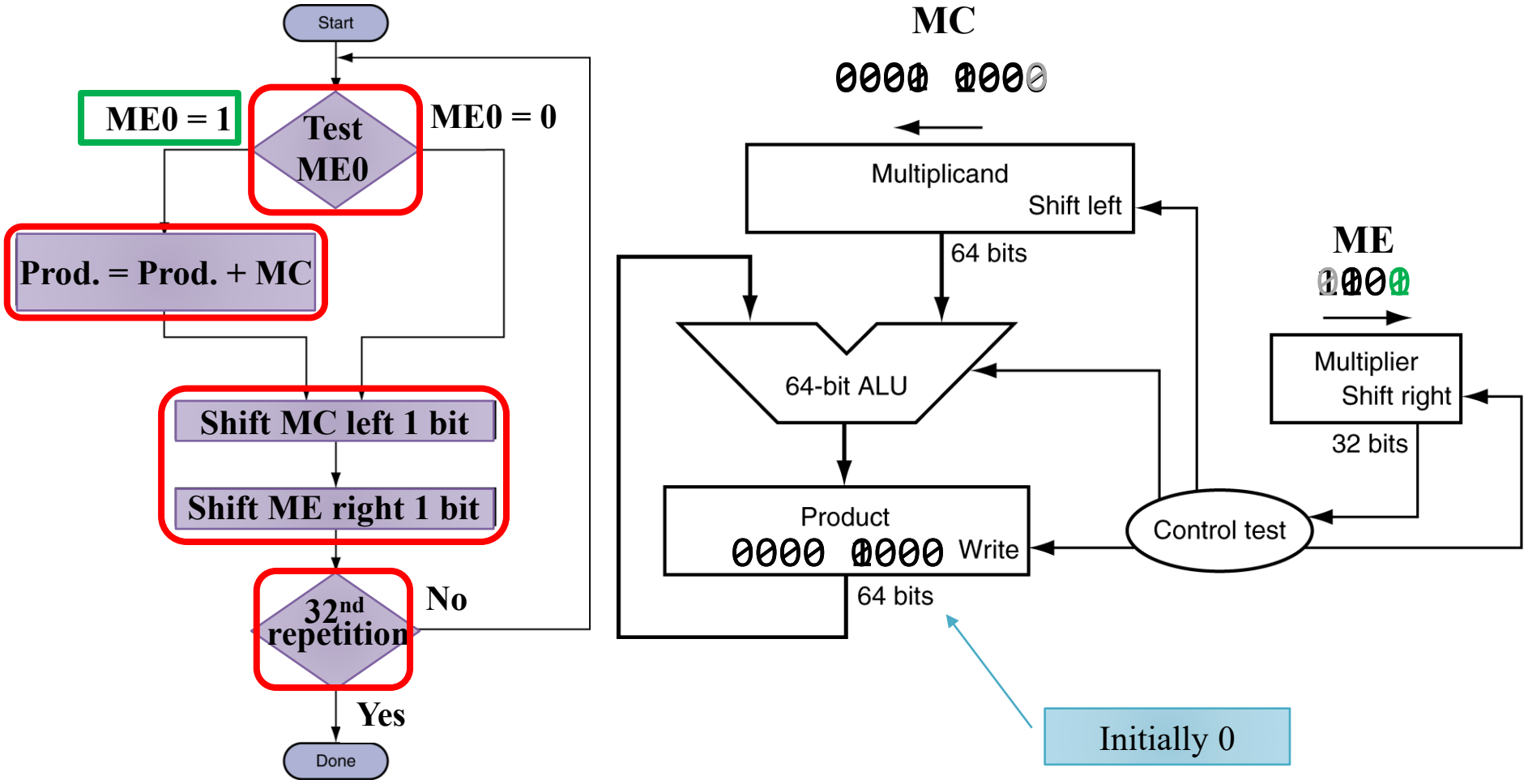
Multiplicand (被乘數)

Multiplier (乘數)

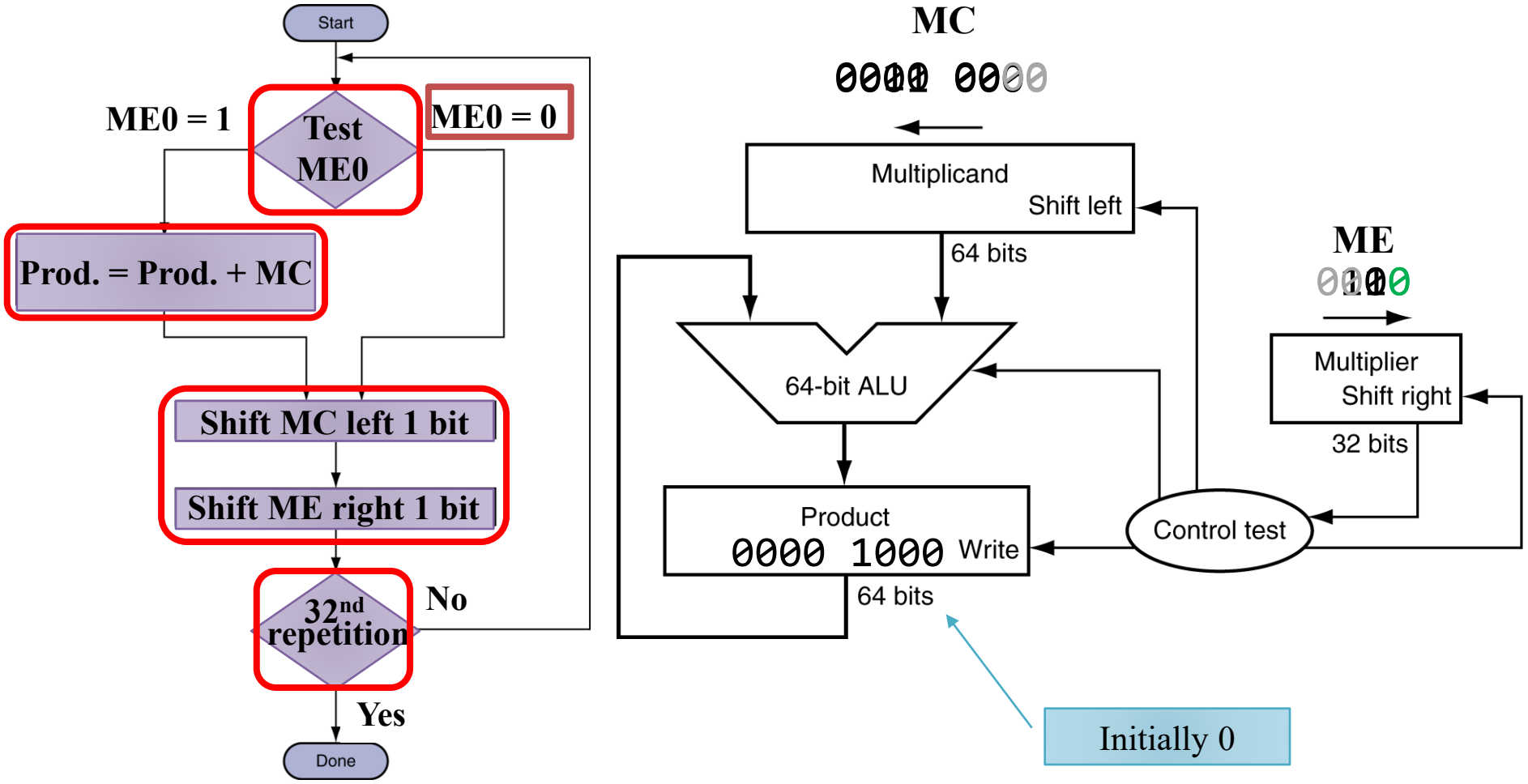
We have 1001 times of 1000



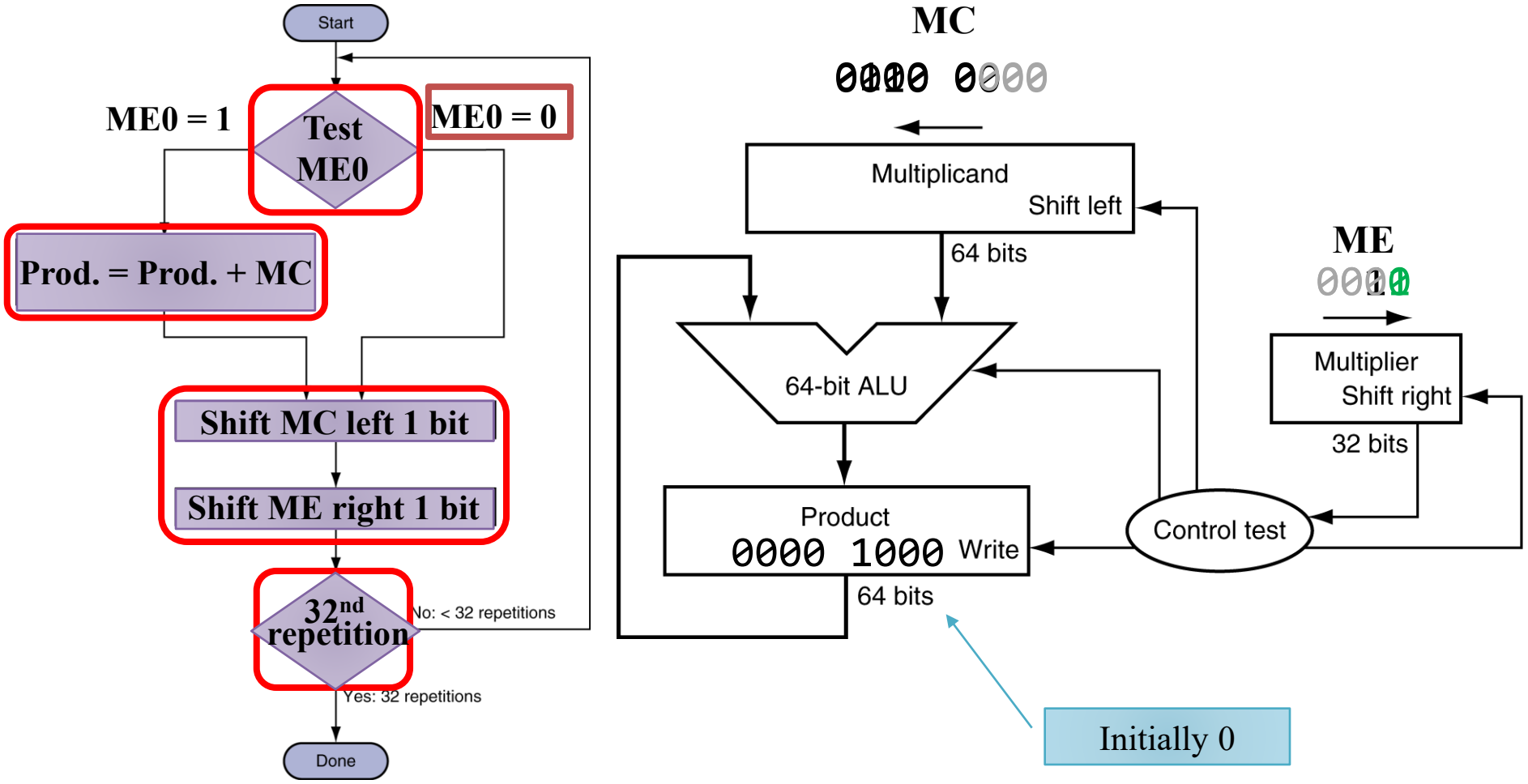
Multiplication Hardware



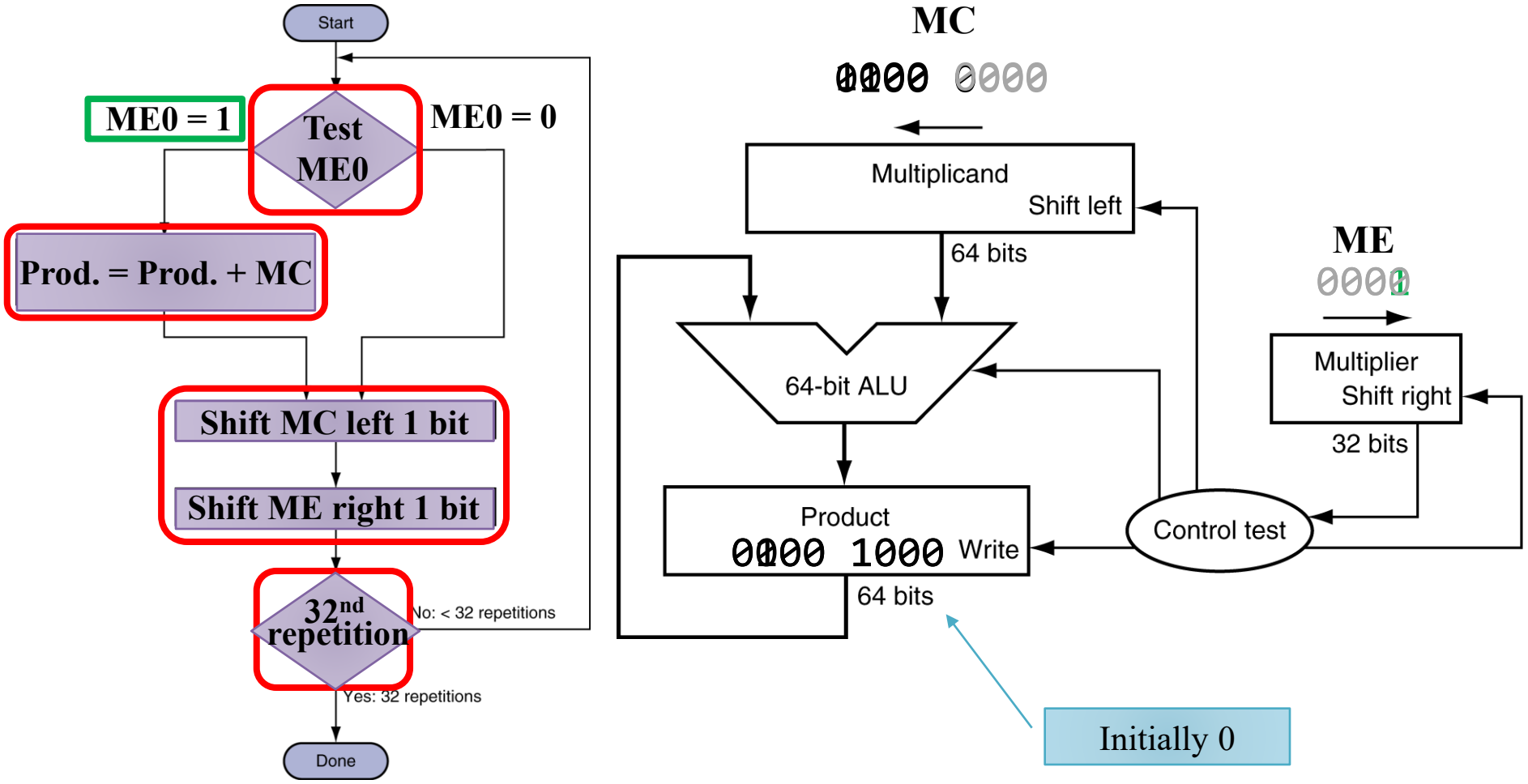
Multiplication Hardware



Multiplication Hardware

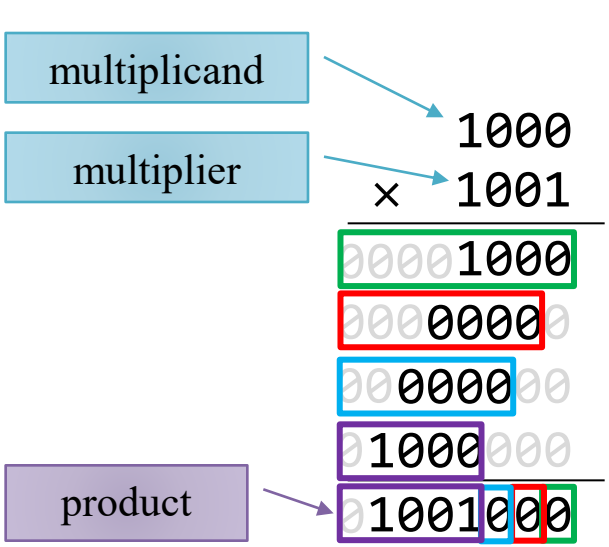


Multiplication Hardware



Optimized Multiplication

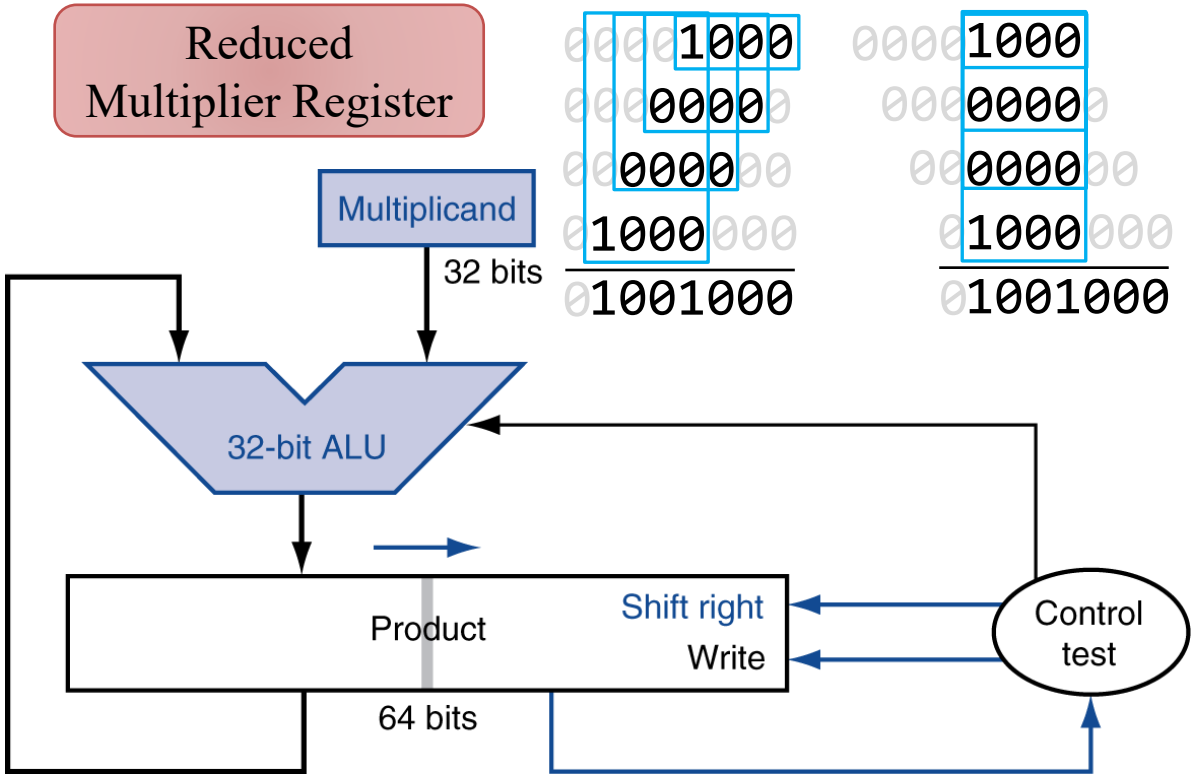
- Start with long-multiplication approach



Length of product is the sum of operand lengths

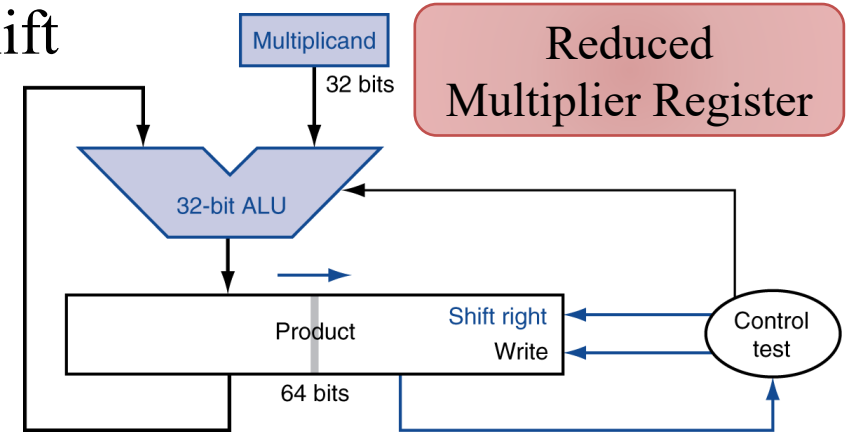
1000 × 1001

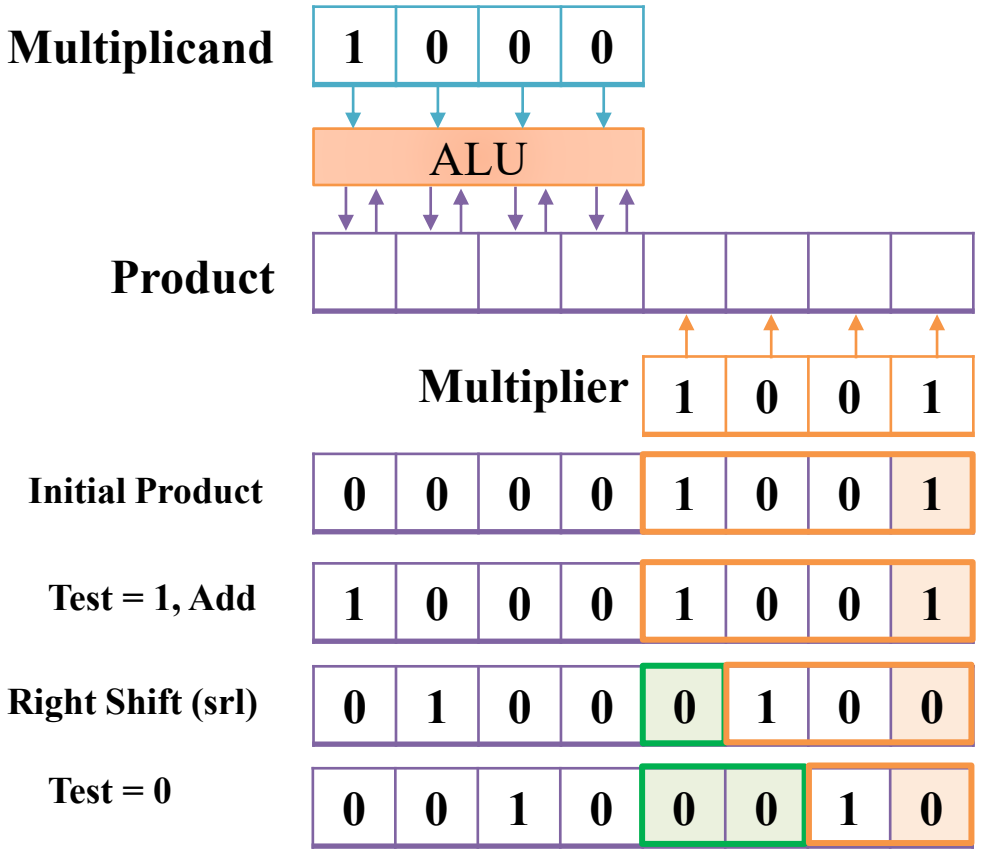
We have 1001 times of 1000

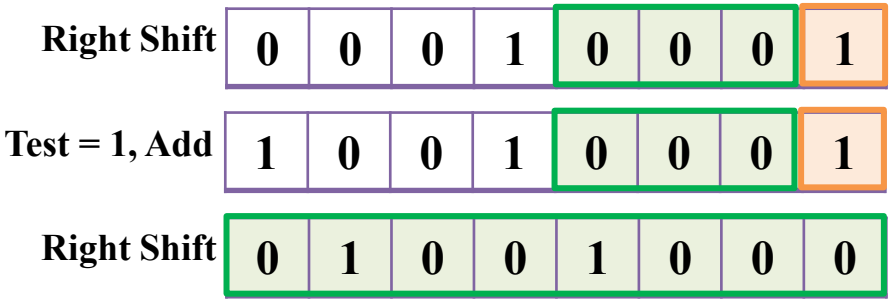


Optimized Multiplier 1000×1001

- Perform steps in parallel: add/shift







Signed Multiplication

- Easiest way
  - Convert multiplicand and multiplier to positive number
  - Remember the original signs
  - In a 64-bit representation, you would have the lower 32-bit product after the previous algorithm completes.

$(-3) \times 2$

1101 0010

Signed $(-3) \times 2 = (-6)$

Unsigned $13 \times 2 = 26$

1111 **1010**

0001 **1010**

8-bit representation

MIPS Multiplication

- Two 32-bit registers for product
  - HI: most-significant 32 bits
  - LO: least-significant 32-bits
- Instructions
  - `mult rs, rt  /  multu rs, rt`
    - 64-bit product in HI/LO
  - `mfhi rd  /  mflo rd`
    - Move from HI/LO to rd
    - Can test HI value to see if product overflows 32 bits
  - `mul rd, rs, rt`
    - Least-significant 32 bits of product –> rd

Special Purpose Registers
PC, HI, LO

Booth's Algorithm Idea

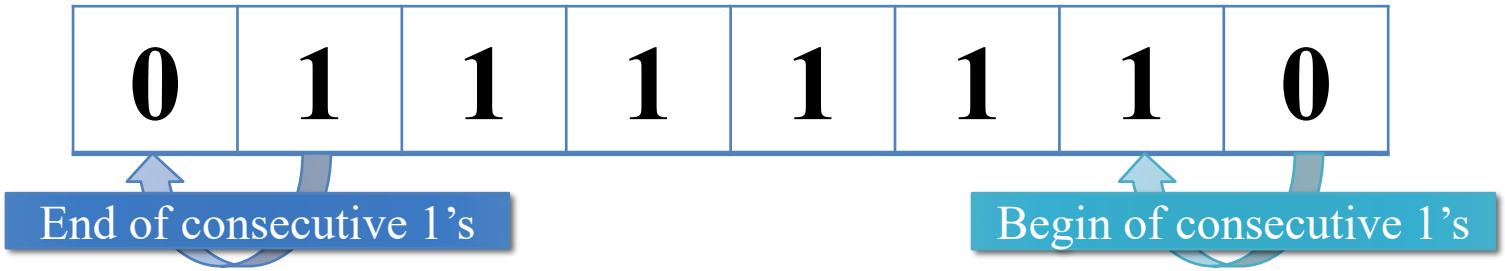
- Dealing with signed multiplication
- Reduce addition process from consecutive 1's

Required 6 addition

$$\text{Multiplicand} \times \quad 01111110$$

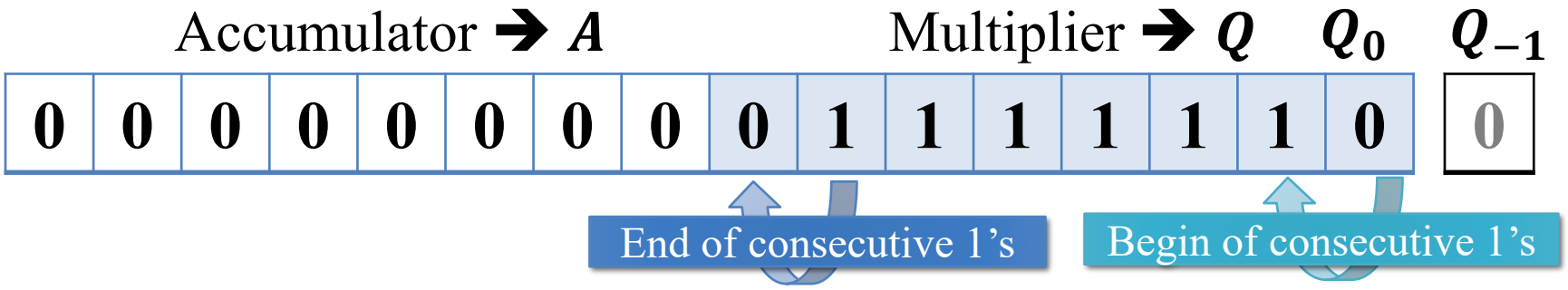
$$= \text{Multiplicand} \times (10000000 - 10)$$

1 addition and 1 subtraction



Booth's Algorithm Rule

Multiplicand (M) × 01111110



Q_0	Q_{-1}	Operator	Meaning
0	0	-	No 1's detected
0	1	$A = A + M$	Begin of 1's
1	0	$A = A - M$	End of 1's
1	1	-	Middle of 1's

Booth's Algorithm Example (1/4)

$7 \times (-5)$

+M: $7_{10} = 00000111_2$
−M: $-7_{10} = 11111001_2$
Q: $-5_{10} = 11111011_2$

Q_0	Q_{-1}	Operator	Meaning
0	0	-	No 1's detected
0	1	$A = A + M$	Begin of 1's
1	0	$A = A - M$	End of 1's
1	1	-	Middle of 1's

	A								Q							Q_0	Q_{-1}
	0	0	0	0	0	0	0	0	1	1	1	1	1	0	1	1	0
Q_0Q_{-1}=10 ➔ A=A-M																	
➔	1	1	1	1	1	0	0	1	1	1	1	1	1	0	1	1	0
Right shift with sign extension																	
➔	1	1	1	1	1	1	0	0	1	1	1	1	1	1	0	1	1
Q_0Q_{-1}=11 ➔ do nothing; right shift with sign extension																	
➔	1	1	1	1	1	1	1	0	0	1	1	1	1	1	1	0	1

Booth's Algorithm Example (2/4)

$7 \times (-5)$

+M: $7_{10} = 00000111_2$
−M: $-7_{10} = 11111001_2$
Q: $-5_{10} = 11111011_2$

Q_0	Q_{-1}	Operator	Meaning
0	0	-	No 1's detected
0	1	$A = A + M$	Begin of 1's
1	0	$A = A - M$	End of 1's
1	1	-	Middle of 1's

	A								Q							Q_0	Q_{-1}
	1	1	1	1	1	1	1	0	0	1	1	1	1	1	1	0	1
Q_0Q_{-1}=01 ➔ A=A+M																	
	0	0	0	0	0	1	0	1	0	1	1	1	1	1	1	0	1
Right shift																	
	0	0	0	0	0	0	1	0	1	0	1	1	1	1	1	1	0
Q_0Q_{-1}=10 ➔ A=A-M																	
	1	1	1	1	1	0	1	1	1	0	1	1	1	1	1	1	0

Booth's Algorithm Example (3/4)

$7 \times (-5)$

+M: $7_{10} = 00000111_2$
−M: $-7_{10} = 11111001_2$
Q: $-5_{10} = 11111011_2$

Q_0	Q_{-1}	Operator	Meaning
0	0	-	No 1's detected
0	1	$A = A + M$	Begin of 1's
1	0	$A = A - M$	End of 1's
1	1	-	Middle of 1's

A								Q							Q_0	Q_{-1}
1	1	1	1	1	0	1	1	1	0	1	1	1	1	1	1	0

Right shift with sign extension

1	1	1	1	1	1	0	1	1	1	0	1	1	1	1	1	1

Q_0Q_{-1}=11 ➔ do nothing; right shift with sign extension

1	1	1	1	1	1	1	0	1	1	1	0	1	1	1	1	1

Q_0Q_{-1}=11 ➔ do nothing; right shift with sign extension

1	1	1	1	1	1	1	1	0	1	1	1	0	1	1	1	1

Booth's Algorithm Example (4/4)

$7 \times (-5)$

+M: $7_{10} = 00000111_2$
−M: $-7_{10} = 11111001_2$
Q: $-5_{10} = 11111011_2$

Q_0	Q_{-1}	Operator	Meaning
0	0	-	No 1's detected
0	1	$A = A + M$	Begin of 1's
1	0	$A = A - M$	End of 1's
1	1	-	Middle of 1's

A								Q							Q_0	Q_{-1}
1	1	1	1	1	1	1	1	0	1	1	1	0	1	1	1	1

Q_0Q_{-1}=11 ➔ do nothing; right shift with sign extension

1	1	1	1	1	1	1	1	1	0	1	1	1	0	1	1	1

Q_0Q_{-1}=11 ➔ do nothing; right shift with sign extension

1	1	1	1	1	1	1	1	1	1	0	1	1	1	0	1	1

1111111111011101_2 –2's complement–
➔Negative $0000000000100011_2 = -35_{10}$

Division

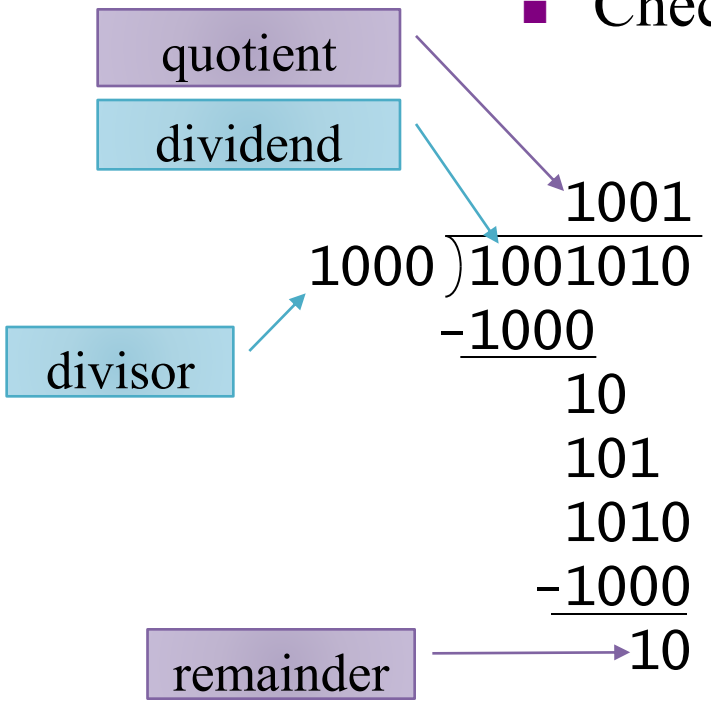
CHAPTER 3: ARITHMETIC FOR COMPUTERS

Division

1001010 ÷ 1000

Dividend (被除數)

Divisor (除數)

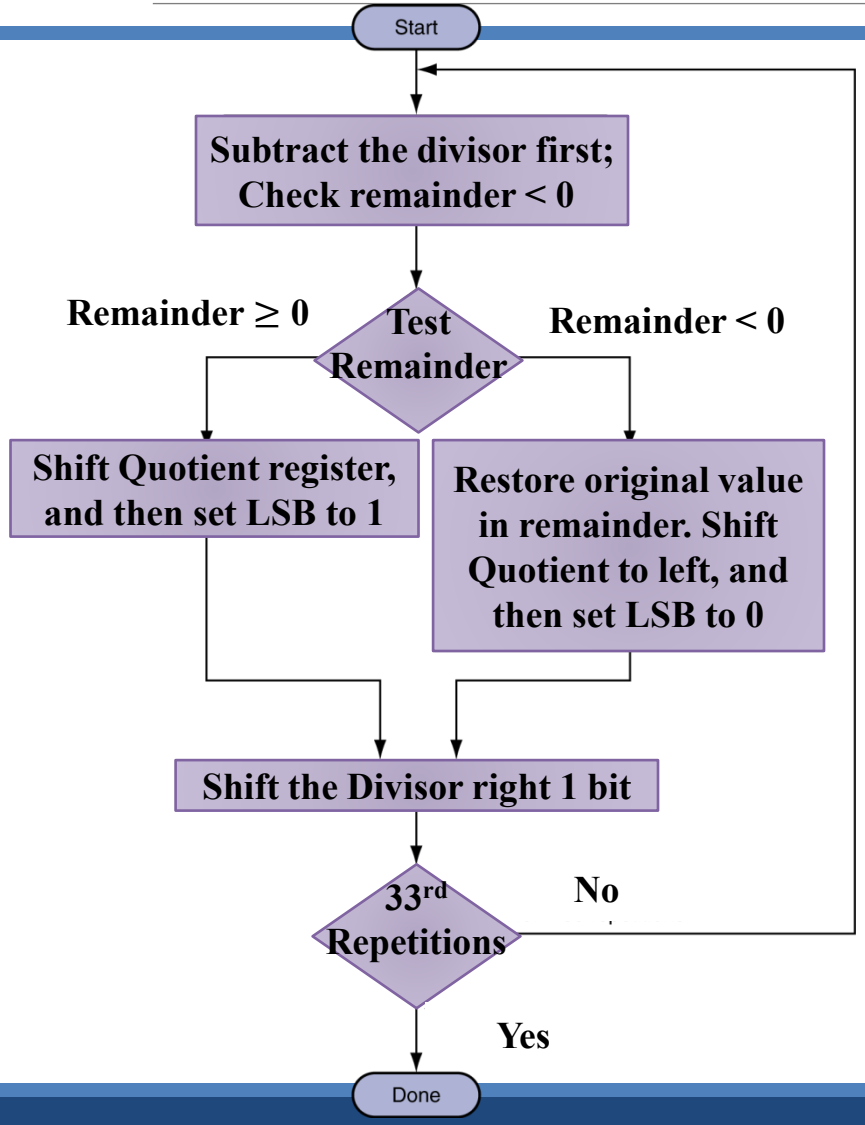


n-bit operands yield *n*-bit quotient and remainder

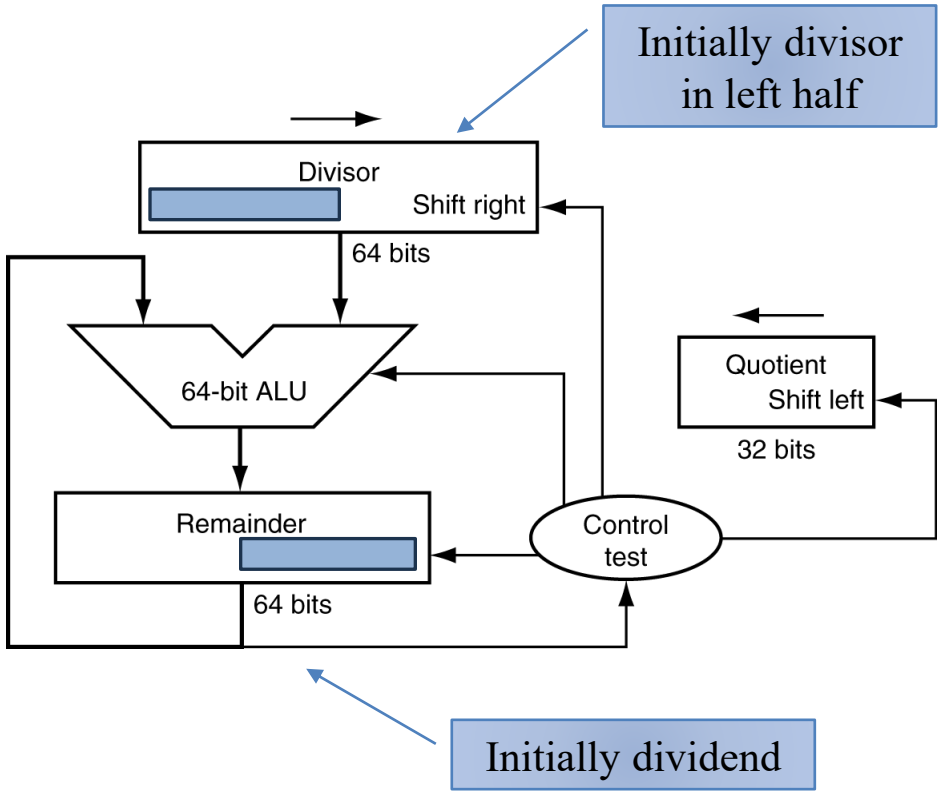
- Check for 0 divisor ➔ not handling in hardware
  - Long Division Approach

1. Compare the current value with divisor
   - If ≥ divisor ➔ Quotient bit = 1; subtract
   - Otherwise ➔ Quotient = 0; keep the value
2. Bring down the next dividend bit
3. Repeat until all dividend bits are processed

Division Hardware

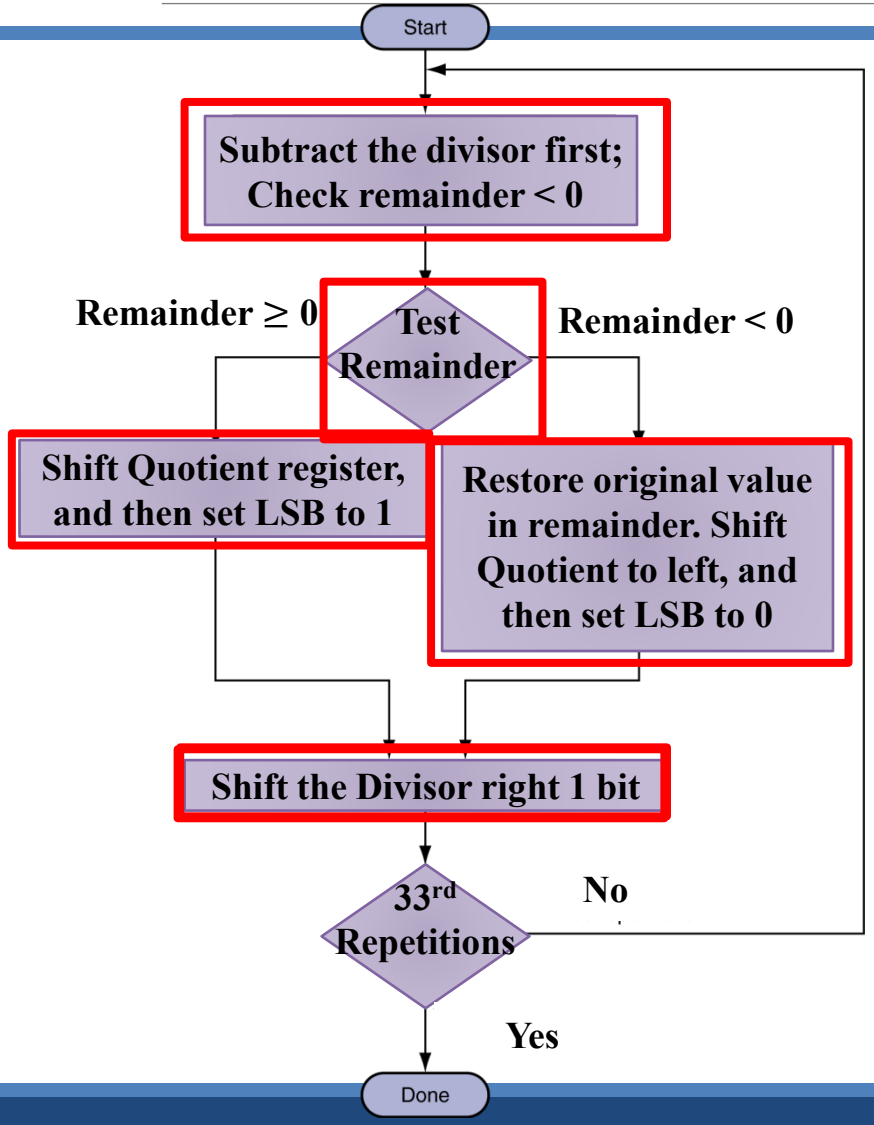


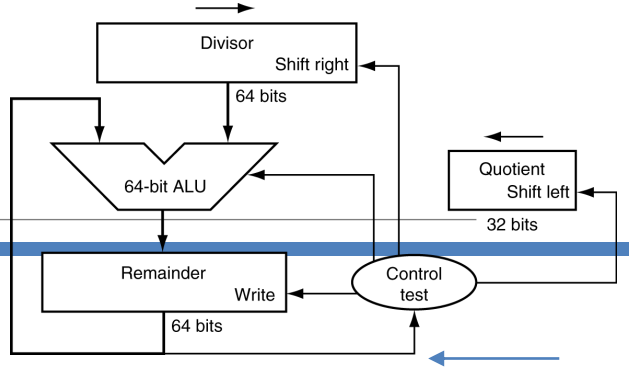
- Restoring division
  Do the subtract; if remainder goes < 0, add divisor back



1010 ÷ 0011

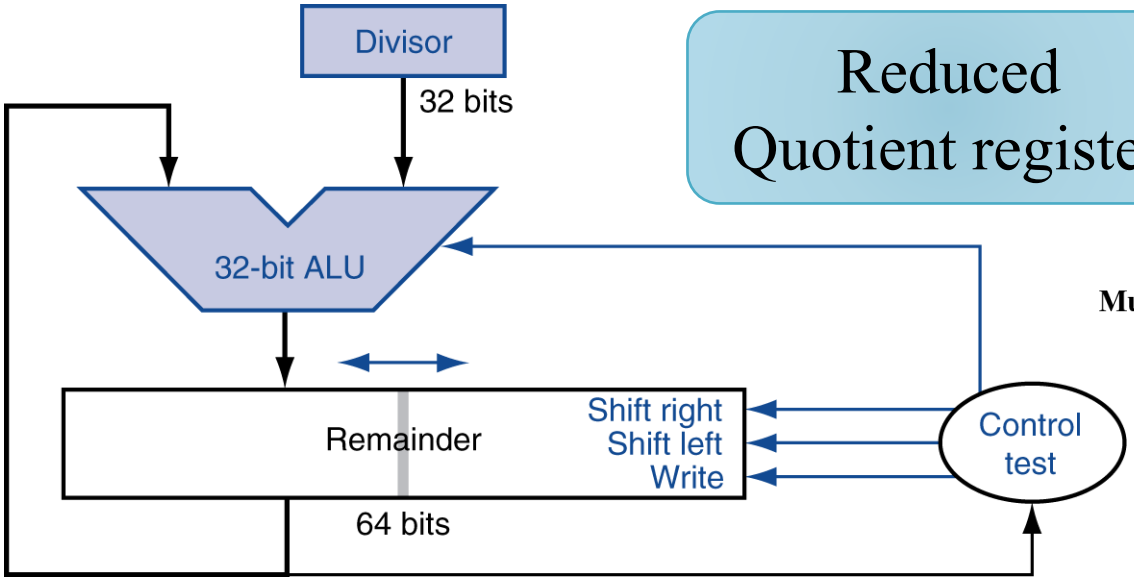
Division Hardware



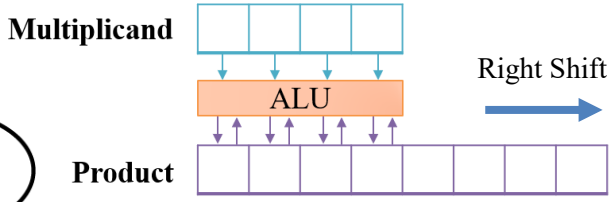


Divisor	Remainder	Quotient
0011 0000	0000 1010	0000
0001 1000	0000 1010	0000
0000 1100	0000 1010	0000
0000 0110	0000 1010	0000
0000 0011	0000 0100	0001
0000 0001	0000 0001	0011

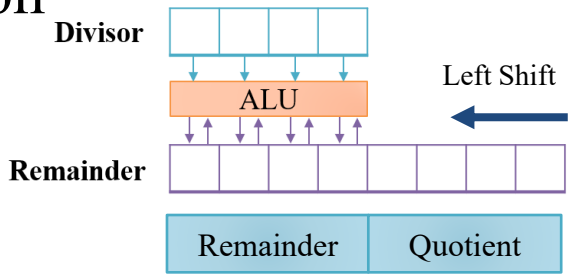
Optimized Divider



Reduced Quotient register



- One cycle per partial-remainder subtraction
- Looks a lot like a multiplier!
  - Same hardware can be used for both



Signed Division

- Simplest method
  - Remember the signs of divisor and dividend
  - Take the absolute magnitude to do the division
  - Then determine the signs of quotient and remainder
    - Make the sign of remainder match the dividend
    - Negate the sign of quotient if the signs of operands are opposite

$$+7 \div +2 \Rightarrow Quotient: +3, Remainder: +1$$

$$-7 \div +2 \Rightarrow Quotient: -3, Remainder: -1$$

$$+7 \div -2 \Rightarrow Quotient: -3, Remainder: +1$$

$$-7 \div -2 \Rightarrow Quotient: +3, Remainder: -1$$

MIPS Division

- Use HI/LO registers for result
  - HI: 32-bit remainder
  - LO: 32-bit quotient
- Instructions
  - div rs, rt / divu rs, rt
  - No overflow or divide-by-0 checking
    - Software must perform checks if required
  - Use mfhi, mflo to access result

Floating Point

CHAPTER 3: ARITHMETIC FOR COMPUTERS

Number Representation

- Integer Number
  - 10,000、10 thousands
  - 1,000,000、1 Million
  - 1,000,000,000、1 Billion
- Real Number
  - 0.1
  - 0.001
  - 0.0001
  - 0.00000000000000456789123
- Normalized Scientific Notation

$$a \times 10^{b}, \qquad where\ a \in \mathbb{R}, \qquad 1 \leq a < 10$$

Floating-Point Representation

- Binary Normalized Scientific Notation

$$1.xxxxxxxxx_2 \times 2^{yyyy}$$

- IEEE-754 Floating-Point Standard

$$(-1)^{Sign} \times (1 + Fraction)_2 \times 2^{Exponent}$$

  - Sign → Positive or Negative
  - Exponent → Power Term
  - Fraction → Significand (有效位數)

Single-precision 32-bit



Double-precision 64-bit



Single-Precision Representation

Single-precision 32-bit

Sign	Exponent	Fraction/Mantissa
1	8-bit	23-bit

Normalized Representation

$$(-1)^{Sign} \times 1.F_2 \times 2^{Exponent}$$

Sign

0 for positive number; 1 for negative number

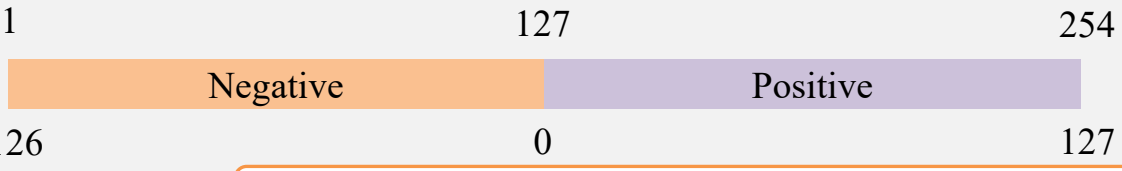
Exponent

Excess-127
RE+127

8-bit ➔ 0000 0000 (0) ~ 1111 1111 (255)

Kept for special usage

IEEE-754



Fraction

Significand; Leading 1 not included

Normalized Number Example

- Dec: 7
- Binary: $111 \rightarrow 1.11 \times 2^{2}$ → Exponent: 2+127=129
- IEEE-754:

Sign	Exponent	Fraction
0	1000 0001	11000000000000000000000

- Dec: 0.5
- Binary: $0.1 \rightarrow 1.0 \times 2^{-1}$ → Exponent: (-1)+127=126
- IEEE-754:

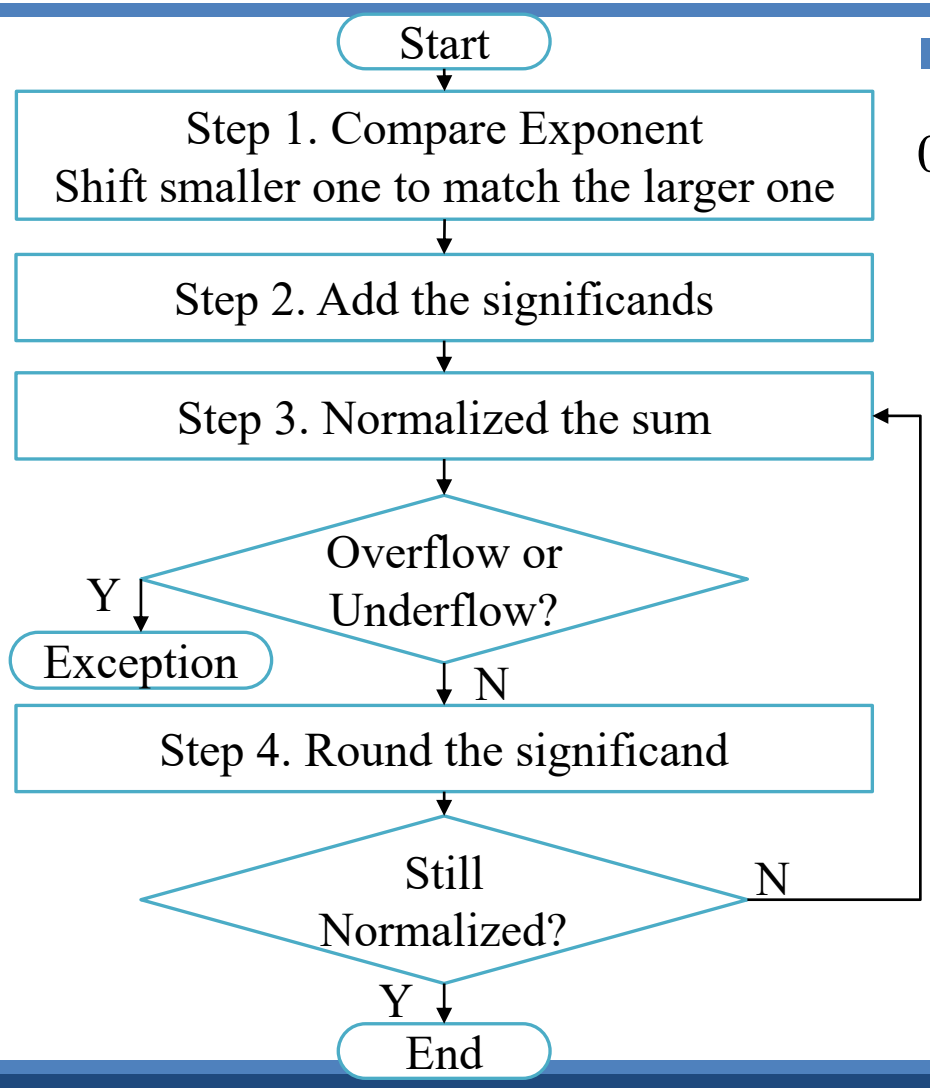
Sign	Exponent	Fraction
0	0111 1110	00000000000000000000000

Easily to compare, especially sorting → Sign → Exponent

Smallest Number $\pm 1.00000000000000000000000_2 \times 2^{-126}$ Underflow → E < -126

Largest Number $\pm 1.11111111111111111111111_2 \times 2^{+127}$ Overflow → E > 127

Floating-Point Addition



- Dec: 0.5
- Dec: -0.4375

$0.1 \rightarrow 1.000 \times 2^{-1}$ $= -7/16_{10} = -7/2^{4}_{10}$

$-0.0111 \rightarrow -1.1100 \times 2^{-2}$

4-bit as the significand precision

- Step 1. Compare Exponent

  $-1.1100 \times 2^{-2} \rightarrow -0.111 \times 2^{-1}$

- Step 2. Add the significand

  $1.0000 \times 2^{-1} + -0.111 \times 2^{-1} = 0.001 \times 2^{-1}$

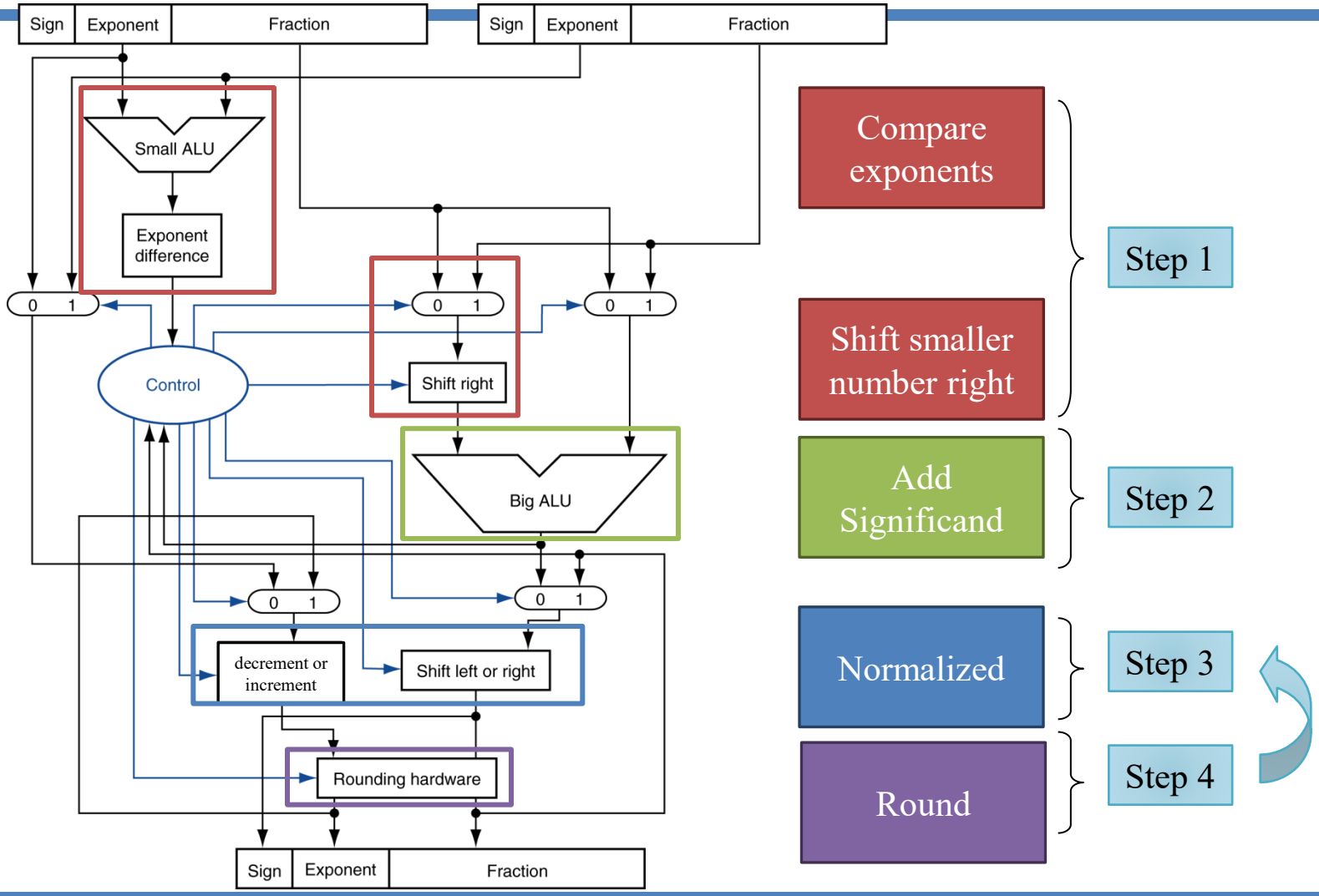
- Step 3. Normalized

  $0.001 \times 2^{-1} = 1.000 \times 2^{-4}$

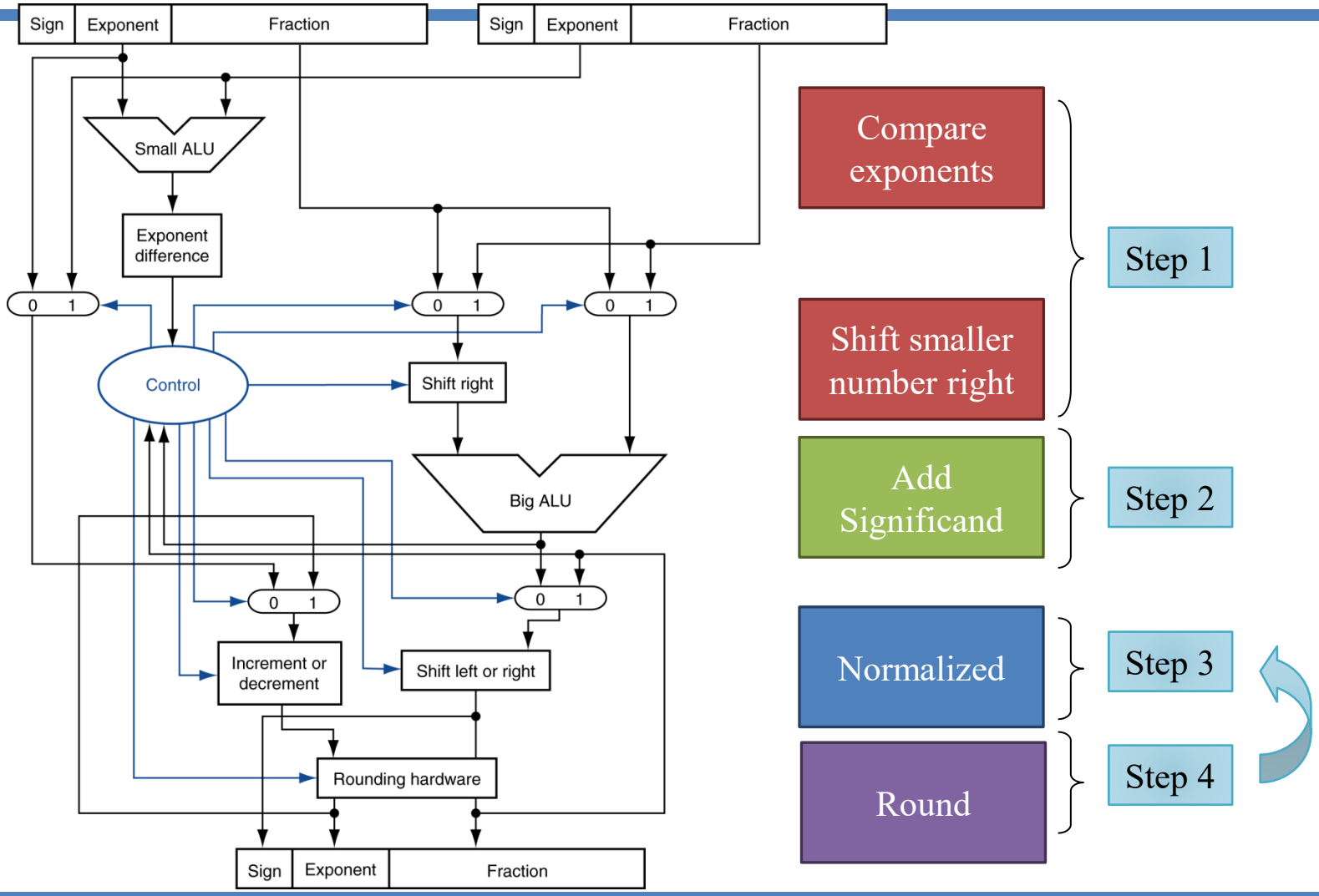
- Step 4. Round ➔ keep 4-bit

  1.000×2^{-4}

FP Adder Hardware



FP Adder Hardware



Normalized $1.F \times 2^E$

Floating-Point Addition - Rounding

Rounding (Round to Nearest, ties to Even)

$F = b_1\ b_2 \ldots b_{22} b_{23}$ | $b_{24}\ b_{25} \ldots\ldots\ b_N$

Available Bits | **Other Bits**

$if\ G = 0 \Rightarrow Truncate$ ➔ Value $< \frac{1}{2}\ b_{23}$

$if\ G = 1$

$\Rightarrow R = 1 \Rightarrow Rounding\ Up$ ➔ Value $> \frac{1}{2}\ b_{23}$

$\Rightarrow R = 0 \Rightarrow S = 0$ ➔ Value $= \frac{1}{2}\ b_{23}$

$\Rightarrow if\ b_{23} = 1 \Rightarrow Rounding\ Up$

$\Rightarrow if\ b_{23} = 0 \Rightarrow Truncate$

*b_{24}: Guard Bit (G)
*b_{25}: Round Bit (R)
*$b_{26} \vee \ldots \vee \ldots \vee b_N$: Sticky Bit (S)

Smallest Normalized Number

$1.0_2 \times 2^{-126}$

$\approx 10^{-38}$

Microsoft

Does all numbers can be represented by this normalized format?

DeNormalized Number Representation

DeNormalized Representation

$$(-1)^{Sign} \times 0.F_2 \times 2^{-126}$$

	Sign	Exponent	Fraction
Denormalized Number		0000 0000	
0		0000 0000	0000 0000 0000 0000 0000 000
infinity		1111 1111	0000 0000 0000 0000 0000 000
NaN Not a number		1111 1111	Non-zero

Smallest Normalized Number

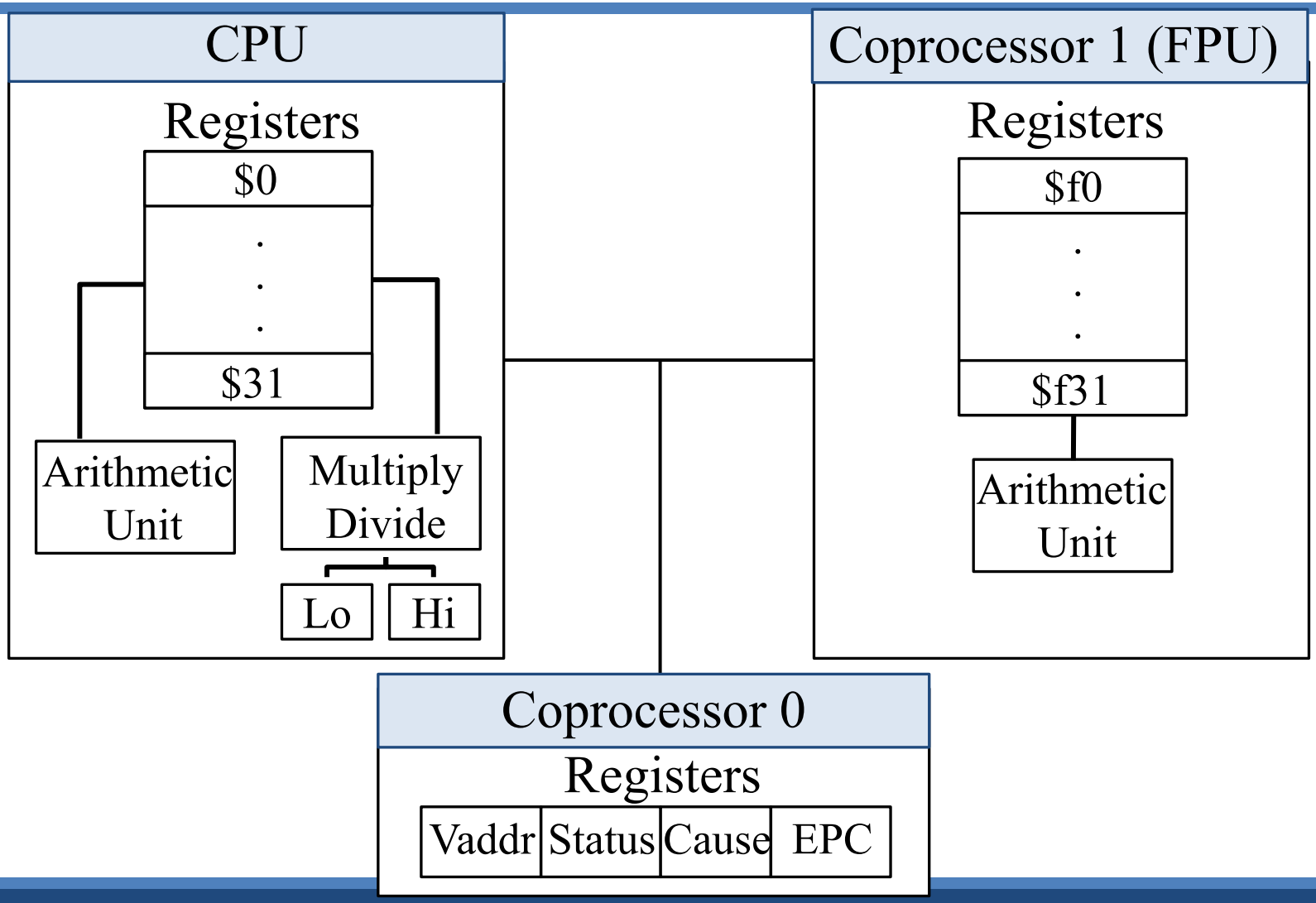
$$1.0_2 \times 2^{-126}$$

Smallest Denormalized Number

$$1.0_2 \times 2^{-23} \times 2^{-126} = 1.0_2 \times 2^{-149}$$



FP Instructions in MIPS



FP Instructions in MIPS

- FP instructions
  - Single precision: `add.s, sub.s, mul.s, div.s`
  - Double precision: `add.d, sub.d, mul.d, div.d`
- Separate FP registers
  - 32 single-precision: `$f0, $f1, … $f31`
  - Paired for double-precision: `$f0/$f1, $f2/$f3`, …
- FP load and store instructions
  - `lwc1, ldc1, swc1, sdc1`
    - e.g., `ldc1 $f8, 32($sp)`

Floating-point Matters

